

Unidad 1: Arquitectura y Diseño.

Tema 1. Arquitectura. Diseño arquitectónico.

Decisiones de diseño arquitectónico. Organización del sistema. Estilos de descomposición modular. Estilos de control. Arquitecturas de referencia. Arquitecturas de sistemas distribuidos. Arquitecturas de aplicaciones.

Los grandes sistemas siempre se descomponen en subsistemas que proporcionan algún conjunto de servicios relacionados. El proceso de diseño inicial que identifica estos subsistemas y establece un marco para el control y comunicación de los subsistemas se llama diseño arquitectónico. El resultado de este proceso de diseño es una descripción de la arquitectura del software.

El proceso de diseño arquitectónico está relacionado con el establecimiento de un marco estructural básico que identifica los principales componentes de un sistema y las comunicaciones entre estos componentes.

Bass señala tres ventajas de diseñar explícitamente y documentar la arquitectura del software:

Comunicación con los stakeholders. La arquitectura constituye una presentación de alto nivel del sistema que puede usarse como punto de discusión por varios stakeholders.

Análisis del sistema. Hacer explícita la arquitectura del sistema en una etapa temprana del desarrollo del sistema requiere realizar algún análisis. Las decisiones de diseño arquitectónico tienen un gran efecto sobre si el sistema puede cumplir los requerimientos críticos tales como rendimiento, fiabilidad y mantenibilidad.

Reutilización a gran escala. Un modelo de arquitectura del sistema es una descripción compacta y manejable de cómo se organiza un sistema y cómo interoperan sus componentes. La arquitectura del sistema es a menudo la misma para sistemas con requerimientos similares y, por lo tanto, pueden soportar reutilización del software a gran escala.

El estilo y estructura particulares elegidos para una aplicación puede, por lo tanto, depender de los requerimientos no funcionales del sistema:

Rendimiento. Si el rendimiento es un requerimiento crítico, la arquitectura debería diseñarse para identificar las operaciones críticas en un pequeño número de subsistemas, con tan poca comunicación como sea posible entre estos subsistemas.

Protección. Si la protección es un requerimiento crítico, debería usarse una arquitectura estructurada en capas, con los recursos más críticos protegidos en las capas más internas y aplicando una validación de seguridad de alto nivel en dichas capas.

Seguridad. Si la seguridad es un requerimiento crítico, la arquitectura debería diseñarse para que las operaciones relacionadas con la seguridad se localizaran en un único subsistema o en un pequeño número de subsistemas. Esto reduce los costes y los problemas de validación de seguridad y hace posible crear los sistemas de protección relacionados con los de seguridad.

Disponibilidad. Si la disponibilidad es un requerimiento crítico, la arquitectura debería diseñarse para incluir componentes redundantes y para que sea posible reemplazar y actualizar componentes sin detener el sistema.

Mantenibilidad. Si la mantenibilidad es un requerimiento crítico, la arquitectura del sistema debería diseñarse usando componentes independientes de grano fino que puedan modificarse con facilidad.

Los diagramas de bloques se usan a menudo para describir diseños de subsistemas en donde cada caja en el diagrama representa un subsistema. Cajas dentro de otras cajas indican que el subsistema se ha descompuesto a su vez en otros subsistemas. Las flechas significan que los datos o señales de control pasan de un subsistema a otro subsistema en la dirección de las flechas.

Bass señala que los diagramas sencillos de cajas y líneas no son representaciones arquitectónicas útiles debido a que no muestran la naturaleza de las relaciones entre los componentes del sistema ni tampoco las propiedades externamente visibles de los componentes. Desde la perspectiva de un diseñador de software, esto es absolutamente correcto. Sin embargo, este tipo de modelo es efectivo para la comunicación con los stakeholders del sistema y para la planificación del proyecto debido a que no muestra los detalles. Los stakeholders pueden relacionarlo con el sistema y tener una visión abstracta del mismo.

El problema general de decidir cómo descomponer un sistema en subsistemas es difícil. Por supuesto, los requerimientos del sistema son un factor fundamental, y debería intentarse crear un diseño en el que hubiera una clara correspondencia entre los requerimientos y los subsistemas. Esto significa que, si los requerimientos cambian, este cambio probablemente esté localizado en un único sitio en vez de distribuido entre varios subsistemas.

Decisiones de diseño arquitectónico

El diseño arquitectónico es un proceso creativo en el que se intenta establecer una organización del sistema que satisfaga los requerimientos funcionales y no funcionales del propio sistema. Debido a que es un proceso creativo, las actividades dentro del proceso difieren radicalmente dependiendo del tipo de sistema a desarrollar, el conocimiento y la experiencia del arquitecto del sistema, y los requerimientos específicos del mismo.

Si bien cada sistema software es único, los sistemas en el mismo dominio de aplicación a menudo tienen arquitecturas similares que reflejan los conceptos fundamentales del dominio. Cuando se diseña la arquitectura de un sistema, se debe decidir qué tiene en común ese sistema con clases de aplicaciones más amplias, y determinar en qué medida el conocimiento de esas arquitecturas de aplicaciones se puede reutilizar.

Las arquitecturas de la mayoría de los sistemas grandes no utilizan un único estilo. Pueden diseñarse diferentes partes del sistema utilizando distintos estilos arquitectónicos. En algunos casos, la totalidad de la arquitectura del sistema puede ser una arquitectura compuesta creada mediante la combinación de diferentes estilos arquitectónicos.

UN estilo arquitectónico incluye las tres siguientes cuestiones de diseño. Se debe elegir la estructura más adecuada, tal como una estructura cliente-servidor o por capas, que permita satisfacer los requerimientos del sistema. Para descomponer las unidades del sistema estructural en módulos, hay que decidir la estrategia para descomponer subsistemas en sus componentes o módulos. Finalmente, en el proceso de modelado de control, se toman decisiones sobre cómo se controla la ejecución de los subsistemas.

El resultado del proceso de diseño arquitectónico es un documento de diseño arquitectónico. Debería describir cómo se estructura el sistema en subsistemas, la aproximación adoptada y cómo se estructura cada subsistema en módulos. Los modelos gráficos del sistema presentan diferentes

perspectivas de la arquitectura. Los modelos arquitectónicos que pueden desarrollarse pueden incluir:

1. Un modelo estructural estático que muestre los subsistemas o componentes que han sido desarrollados como unidades separadas.
2. Un modelo de proceso dinámico que muestre cómo se organiza el sistema en procesos en tiempo de ejecución. Este modelo puede ser diferente del modelo estático.
3. Un modelo de interaz: que defina los servicios ofrecidos por cada subsistema a través de su interfaz pública.
4. Modelos de relaciones que muestren las relaciones. tales como el flujo de datos. entre los subsistemas.
5. Un modelo de distribución, que muestre cómo se distribuyen los subsistemas entre las computadoras.

Organización del sistema

La organización de un sistema refleja la estrategia básica usada para estructurar dicho sistema. Deben tomarse decisiones sobre la totalidad del modelo organizacional de un sistema al principio del proceso de diseño arquitectónico.

un estilo de repositorio de datos, un estilo de servicios y servidores compartidos y una máquina abstracta o estilo por capas en donde el sistema se organiza en un conjunto de capas funcionales

Los tres estilos organizacionales ampliamente usados: un estilo de repositorio de datos, un estilo de servicios y servidores compartidos y una máquina abstracta o estilo por capas en donde el sistema se organiza en un conjunto de capas funcionales.

El modelo de repositorio

Los subsistemas que forman un sistema deben intercambiar información para que puedan trabajar conjuntamente de forma efectiva. Esto es puede conseguir de dos formas fundamentales:

1. Todos los datos compartidos se almacenan en una base de datos central a la que puede acceder por todos los subsistemas. Un modelo de sistema basado en una base de datos compartida se denomina algunas veces modelo de repositorio.
2. Cada subsistema mantiene su propia base de datos. Los datos se intercambian con otros subsistemas mediante el paso de mensajes entre ellos.



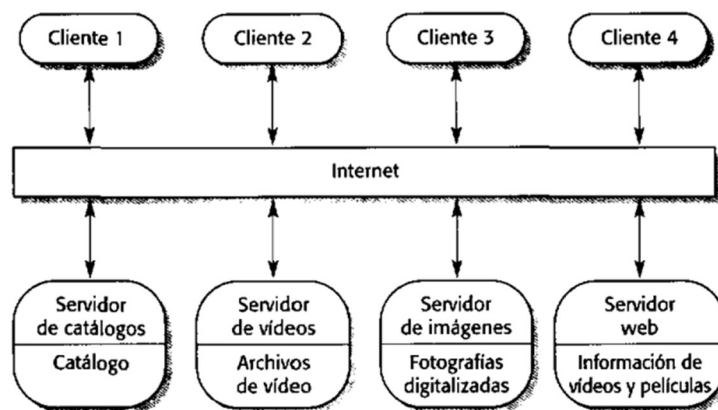
Las ventajas y desventajas de un repositorio compartido son las siguientes:

1. Es una forma eficiente de compartir grandes cantidades de datos. No hay necesidad de transmitir datos explícitamente de un subsistema a otro.
2. Sin embargo, los subsistemas deben estar acordes con el modelo de datos del repositorio. Inevitablemente, hay un compromiso entre las necesidades específicas de cada herramienta. El rendimiento puede verse afectado de forma adversa por este compromiso. Puede ser difícil o imposible integrar nuevos subsistemas si sus modelos de datos no se ajustan al esquema acordado.
3. Los subsistemas que producen datos no necesitan conocer cómo se utilizan sus datos por otros subsistemas.
4. Sin embargo, la evolución puede ser difícil a medida que se genera un gran volumen de información de acuerdo con el modelo de datos establecido. Traducir esto a un nuevo modelo, desde luego, tendrá un coste elevado; puede ser difícil o incluso imposible.

El modelo cliente-servidor

El modelo arquitectónico cliente-servidor es un modelo de sistema en el que dicho sistema se organiza como un conjunto de servicios y servidores asociados, más unos clientes que acceden y usan los servicios.

Los clientes pueden conocer los nombres de los servidores disponibles y los servicios que éstos proporcionan. Sin embargo, los servidores no necesitan conocer la identidad de los clientes o cuántos clientes tienen. Los clientes acceden a los servicios proporcionados por un servidor a través de llamadas a procedimientos remotos usando un protocolo de petición-respuesta tal como el protocolo http usado en la WWW.



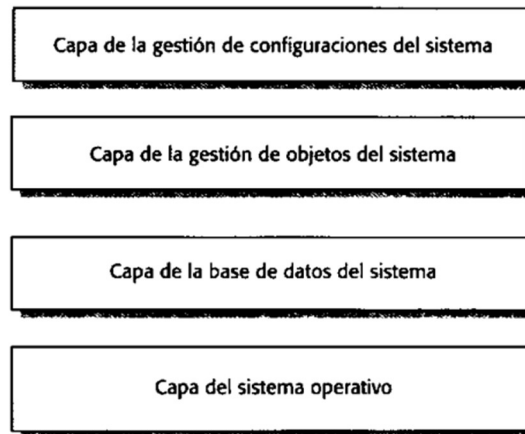
La ventaja más importante del modelo cliente-servidor es que es una arquitectura distribuida, Se puede hacer un uso efectivo de los sistemas en red con muchos procesadores distribuidos, Es fácil añadir un nuevo servidor e integrarlo con el resto del sistema o actualizar los servidores de forma transparente sin afectar al resto del sistema.

Sin embargo, puede ser necesario realizar cambios a los clientes y servidores existentes para obtener los mayores beneficios de la integración de un nuevo servidor. Puede no haber un modelo de datos compartidos entre los servidores, y los subsistemas pueden organizar sus datos de formas diferentes. Esto significa que los modelos de datos específicos pueden establecerse para cada servidor con el fin de optimizar su rendimiento.

El modelo de capas

El modelo de capas de una arquitectura (algunas veces denominada modelo de máquina abstracta) organiza el sistema en capas, cada una de las cuales proporciona un conjunto de servicios. Cada capa puede pensarse como una máquina abstracta cuyo lenguaje máquina se define por los servicios proporcionados por la capa.

La aproximación por capas soporta el desarrollo incremental de sistemas. A medida que se desarrolla una capa, algunos de los servicios proporcionados por esa capa pueden estar disponibles para los usuarios. Esta arquitectura también soporta bien los cambios y es portable. En la medida en la que su interfaz permanezca sin cambios, una capa puede reemplazarse por otra capa equivalente.



Estilos de descomposición modular

Después de que se haya elegido la organización del sistema en su totalidad, es necesario decidir la aproximación a usar para descomponer los subsistemas en módulos. No existe una distinción rígida entre la organización del sistema y la descomposición modular.

No hay una distinción clara entre subsistemas y módulos, pero resulta útil pensar sobre ellos de la siguiente forma:

1. Un subsistema es un sistema en sí mismo, cuyo funcionamiento no depende de los servicios proporcionados por otros subsistemas. Los subsistemas se componen de módulos y tienen interfaces definidas, las cuales se usan para comunicarse con otros subsistemas.
2. Un módulo es normalmente un componente de un subsistema que proporciona uno o más servicios a otros módulos. A su vez éste usa los servicios proporcionados por otros módulos. Esto no se suele considerar como un sistema independiente. Los módulos se componen normalmente de varios componentes del sistema más simples.

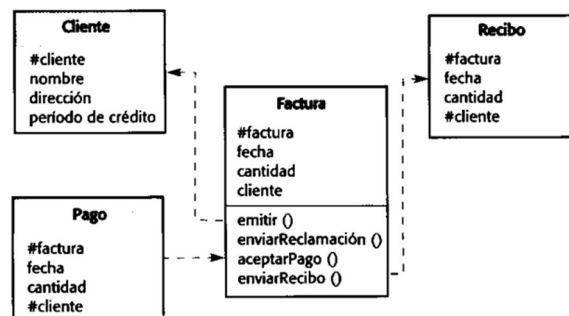
Hay dos estrategias principales que se pueden usar cuando se descomponga un subsistema en módulos:

1. **Descomposición orientada a objetos.** en la que se descompone un sistema en un conjunto de objetos que se comunican.
2. **Descomposición orientadas al flujo de funciones.** en la que se descompone un sistema en módulos funcionales que aceptan datos y los transforman en datos de salida.

En la aproximación orientada a objetos, los módulos son objetos con estado privado y operaciones definidas sobre ese estado. En el modelo de flujos de funciones, los módulos son transformaciones funcionales. En ambos casos, los módulos pueden implementarse como componentes secuenciales o como procesos.

Descomposición orientada a objetos

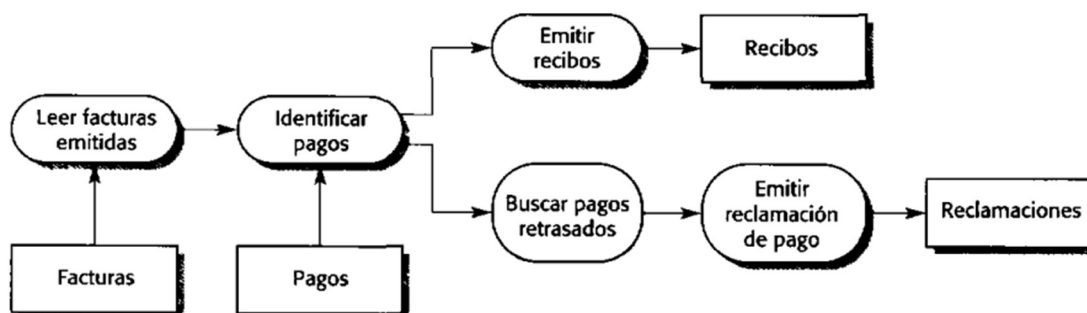
Una descomposición orientada a objetos está relacionada con las clases de objetos, sus atributos y sus operaciones. Cuando se implementa, los objetos se crean a partir de estas clases y se usan algunos modelos de control para coordinar las operaciones de los objetos. En este ejemplo particular, la clase Factura tiene varias operaciones asociadas que implementan la funcionalidad del sistema. Esta clase hace uso de otras clases que representan a los clientes, a los pagos y a los recibos.



Las ventajas de la aproximación orientada a objetos son bien conocidas. Debido a que los objetos están débilmente acoplados, la implementación de los objetos puede modificarse sin afectar a otros objetos. Sin embargo, la aproximación orientada a objetos tiene desventajas. Para utilizar los servicios, los objetos deben referenciar de forma explícita el nombre y la interfaz de otros objetos. Si se requiere un cambio de interfaz para satisfacer los cambios del sistema propuestos, se debe evaluar el efecto de ese cambio sobre todos los usuarios de los objetos cambiados.

Descomposición orientada a flujos de funciones

En una descomposición orientada a flujos de funciones o modelo de flujo de datos, las transformaciones funcionales procesan sus entradas y producen salidas. Los datos fluyen de una función a otra y se transforman a medida que se mueven a través de la secuencia de funciones. Cada paso de procesamiento se implementa como una transformación. Los datos de entrada fluyen a través de estas transformaciones hasta que se convierten en datos de salida. Las transformaciones se pueden ejecutar secuencialmente o en paralelo. Los datos pueden ser procesados por cada transformación elemento a elemento o en un único lote.



Las ventajas de esta arquitectura son las siguientes:

1. Permite la reutilización de transformaciones.
2. Es intuitiva puesto que muchas personas piensan en su trabajo en términos de procesamiento de entradas y salidas.
3. Generalmente se puede hacer evolucionar de forma directa el sistema añadiendo nuevas transformaciones.
4. Es sencilla de implementar ya sea como un sistema concurrente o como uno secuencial.

El principal problema con este estilo es que tiene que haber un formato común para transferir los datos de forma que puedan ser reconocidos por todas las transformaciones. Cada transformación debe estar acorde con las transformaciones con las que se comunica sobre el formato de los datos a procesar o bien se debe imponer un formato estándar para todos los datos comunicados. Esto último es la única aproximación factible cuando las transformaciones son independientes y reutilizables.

Estilos de control

Los modelos para estructurar un sistema están relacionados con la forma en que un sistema se descompone en subsistemas. Para trabajar como un sistema, los subsistemas deben ser controlados para que sus servicios se entreguen en el lugar correcto en el momento preciso. Los modelos de control a nivel arquitectónico están relacionados con el flujo de control entre subsistemas. Hay dos estilos de control genéricos que se usan en sistemas software:

Control centralizado: Un subsistema tiene toda la responsabilidad para controlar e iniciar y detener a otros subsistemas. También puede devolver el control a otro subsistema, pero esperará que le sea devuelta la responsabilidad del control.

Control basado en eventos. En lugar de que la información de control esté embebida en un subsistema, cada subsistema puede responder a eventos generados externamente. Estos eventos podrían provenir de otros subsistemas o del entorno del sistema.

Control centralizado

En un modelo de control centralizado, un subsistema se diseña como el controlador del sistema y tiene la responsabilidad de gestionar la ejecución de otros subsistemas. Los modelos de control centralizado se dividen en dos clases, según que los subsistemas controlados se ejecuten secuencialmente o en paralelo.

El modelo de llamada-retorno. Es el modelo usual de subrutina descendente en donde el control comienza al inicio de una jerarquía de subrutinas y, a través de las llamadas a subrutinas, el control pasa a niveles inferiores en el árbol de la jerarquía. El modelo de subrutinas solamente es aplicable a sistemas secuenciales.

El modelo del gestor. Éste es aplicable a sistemas concurrentes. Un componente del sistema se diseña como un gestor del sistema y controla el inicio, parada y coordinación del resto de los procesos del sistema. Un proceso es un subsistema o módulo que puede ejecutarse en paralelo con otros procesos.

Sistemas dirigidos por eventos

En los modelos de control centralizados, las decisiones de control se determinan normalmente por los valores de algunas variables de estado del sistema. Por el contrario, los modelos de control dirigidos por eventos se rigen por eventos generados externamente. El término evento en este contexto no significa una señal binaria. Puede ser una señal dentro de un rango de valores o una entrada de un comando desde un menú. La diferencia entre un evento y una entrada simple es que la aparición del evento está fuera del control del proceso que maneja ese evento.

Modelos de control dirigidos por eventos:

Modelos de transmisión (broadcast). En estos modelos, un evento se transmite a todos los subsistemas. Cualquier subsistema que haya sido programado para manejar ese evento puede responder a él.

Modelos dirigidos por interrupciones. Éstos se usan exclusivamente en sistemas de tiempo real, en donde las interrupciones externas son detectadas por un manejador de interrupciones. A continuación, éstas se envían a algún otro componente para su procesamiento.

En un modelo de transmisión, los subsistemas registran un interés en eventos específicos. Cuando estos eventos ocurren, el control se transfiere al subsistema que puede manejar el evento. La diferencia entre este modelo y el modelo centralizado es que la política de control no está embebida en el manejador de eventos y mensajes. Los subsistemas deciden qué eventos requieren y el manejador de eventos y mensajes asegura que estos eventos sean enviados a dichos subsistemas.

Todos los eventos podrían enviarse a todos los subsistemas, pero esto supondría una gran sobrecarga de procesamiento. A menudo, el manejador de eventos y de mensajes mantiene un registro de los subsistemas y de los eventos que a éstos les interesan.

La ventaja de esta aproximación de transmisión (Broadcast) es que la evolución es relativamente simple. Un nuevo subsistema puede integrarse para manejar clases particulares de eventos registrando sus eventos con el manejador de eventos. Cualquier subsistema puede activar otros subsistemas sin conocer su nombre o localización. Los subsistemas pueden implementarse sobre máquinas distribuidas. Esta distribución es transparente para el resto de los subsistemas.

La desventaja de este modelo es que los subsistemas no conocen si los eventos se manejarán ni cuándo serán manejados. Cuando un subsistema genera un evento no conoce qué subsistemas han registrado un interés en dicho evento. Es bastante probable que subsistemas diferentes se registren para los mismos eventos. Esto puede ocasionar conflictos cuando están disponibles los resultados del manejo del evento.



Este modelo se usa principalmente en sistemas de tiempo real en los que es necesaria una respuesta inmediata a algún evento. Puede ser combinada con el modelo de gestión centralizado. El gestor central maneja la ejecución normal del sistema con un control para emergencias basado en interrupciones.

Arquitecturas de referencia

Los modelos arquitectónicos citados anteriormente son modelos generales: pueden aplicarse a muchas clases de aplicaciones. Al igual que los modelos generales, también pueden usarse los modelos arquitectónicos que son específicos para un dominio particular de aplicación. Si bien las instancias de estos sistemas difieren en los detalles, la estructura arquitectónica común puede reutilizarse cuando se desarrollan nuevos sistemas. Estos modelos arquitectónicos se denominan arquitecturas de dominio específico.

Hay dos tipos de modelos arquitectónicos de dominio específico:

Modelos genéricos. Son abstracciones obtenidas a partir de varios sistemas reales. Encapsulan las características principales de estos sistemas. Por ejemplo, en sistemas de tiempo real, podría haber modelos arquitectónicos genéricos de diferentes tipos de sistemas tales como sistemas de recolección de datos o sistemas de monitorización.

Modelos de referencia. Son más abstractos y describen una clase más amplia de sistemas. Constituyen un modo de informar a los diseñadores sobre la estructura general de esta clase de sistemas. Los modelos de referencia normalmente se obtienen a partir de un estudio del dominio de la aplicación. Representan una arquitectura ideal que incluye todas las características que los sistemas podrían incorporar.

Los modelos genéricos también pueden servir como modelos de referencia. Hacemos aquí la distinción entre ellos debido a que los modelos genéricos pueden reutilizarse directamente en un diseño. Los modelos de referencia se usan normalmente para comunicar conceptos del dominio y comparar o evaluar posibles arquitecturas. Las arquitecturas de referencia normalmente no se consideran como un camino para la implementación. En su lugar, su principal función es una forma de tratar arquitecturas específicas del dominio y de comparar sistemas diferentes en un dominio. Un modelo de referencia proporciona un vocabulario para realizar comparaciones. Dicho modelo actúa como una base, frente a la cual los sistemas pueden ser evaluados.

Los sistemas de aplicaciones intentan adecuarse a necesidades organizacionales o de negocio. Todos los negocios tienen mucho en común -necesitan contratar a personas, emitir facturas, mantener la contabilidad y así sucesivamente- y esto es especialmente cierto para negocios que operan en el mismo sector. Por lo tanto, además de las funciones de negocio generales, todas las compañías telefónicas necesitan sistemas para conectar las llamadas, gestionar su red, emitir facturas a los clientes, etcétera. Como consecuencia, las aplicaciones de los sistemas que utilizan estos negocios tienen también mucho en común.

Arquitectura de aplicaciones

Los sistemas de aplicaciones intentan adecuarse a necesidades organizacionales o de negocio. Todos los negocios tienen mucho en común -necesitan contratar a personas, emitir facturas, mantener la contabilidad y así sucesivamente- y esto es especialmente cierto para negocios que operan en el mismo sector. Por lo tanto, además de las funciones de negocio generales, todas las compañías telefónicas necesitan sistemas para conectar las llamadas, gestionar su red, emitir facturas a los clientes, etcétera. Como consecuencia, las aplicaciones de los sistemas que utilizan estos negocios tienen también mucho en común.

Se puede usar arquitecturas de aplicaciones genéricas de varias formas:

Como un punto de partida para el proceso de diseño arquitectónico. Si no está familiarizado con este tipo de aplicación, puede basar sus diseños iniciales sobre arquitecturas genéricas.

Como una lista de comprobación de un diseño. Si ha desarrollado un diseño arquitectónico de un sistema, puede comprobarlo contrastándolo con la arquitectura de aplicación genérica para ver si ha omitido algún componente de diseño importante.

Como una forma de organizar el trabajo del grupo de desarrollo. La arquitectura de la aplicación identifica características estructurales estables de las arquitecturas de los sistemas y, en muchos casos, es posible desarrollarlas en paralelo. Usted puede asignar trabajo a miembros del grupo para implementar diferentes subsistemas dentro de la arquitectura.

Como una forma de evaluar los componentes para su reutilización. Si usted tiene componentes, podría ser capaz de reutilizarlos; puede comparar éstos con las estructuras genéricas para ver si es probable la reutilización en la aplicación que está desarrollando.

Como un vocabulario para hablar sobre tipos de aplicaciones. Si está tratando una aplicación específica o intentando comparar aplicaciones del mismo tipo, entonces puede usar los conceptos identificados en la arquitectura genérica para hablar de las aplicaciones.

Cuando se examina la organización arquitectónica de las aplicaciones, muchas aplicaciones aparentemente distintas tienen mucho en común. Ilustramos esto describiendo las arquitecturas de cuatro grandes tipos de aplicaciones

Aplicaciones de procesamiento de datos. Las aplicaciones de procesamiento de datos son aplicaciones conducidas por los datos. Procesan datos por lotes sin intervenciones explícitas del usuario durante el procesamiento. Los sistemas de procesamiento por lotes se usan normalmente en aplicaciones de negocio en donde se realizan operaciones similares sobre grandes cantidades de datos. Dichas aplicaciones manejan un amplio rango de funciones administrativas tales como nóminas, facturación, contabilidad y publicidad.

Aplicaciones de procesamiento de transacciones. Las aplicaciones de procesamiento de transacciones son aplicaciones centradas en bases de datos que procesan peticiones del usuario para obtener información y para actualizar la información en una base de datos. Éstos son los tipos más comunes de sistemas de negocio interactivos. Se organizan de forma que las acciones del usuario no pueden interferir entre sí y se mantenga la integridad de la base de datos. Esta clase de sistema incluye sistemas bancarios interactivos, sistemas de comercio electrónico, sistemas de información y sistemas de reservas.

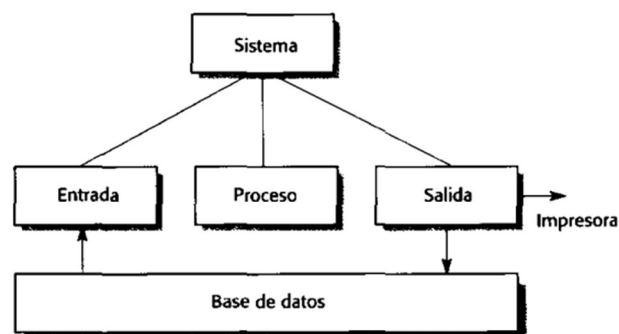
Sistemas de procesamiento de eventos. Ésta es una clase muy amplia de aplicaciones en las que las acciones del sistema dependen de la interpretación de eventos en el entorno del sistema. Estos eventos podrían ser la entrada de una orden por un usuario del sistema o un cambio en las variables que son monitorizadas por el sistema. Muchas aplicaciones basadas en PCs, entre las que se incluyen juegos, sistemas de edición tales como procesadores de texto, hojas de cálculo, editores de imágenes y sistemas de presentación son sistemas de procesamiento de eventos. Los sistemas de tiempo real también se encuentran dentro de esta categoría.

Sistemas de procesamiento de lenguajes. Los sistemas de procesamiento de lenguajes son sistemas en los que las intenciones del usuario se expresan en un lenguaje formal (como, por ejemplo, Java). Los sistemas de procesamiento de lenguajes procesan este lenguaje en algún formato interno y entonces interpretan su representación interna. Los sistemas de procesamiento de lenguajes más conocidos son los compiladores, que traducen programas de lenguajes de alto nivel a código máquina. Sin embargo, los sistemas de procesamiento de lenguajes se utilizan también para interpretar lenguajes de comandos para bases de datos, sistemas de información y lenguajes de marcado tales como XML (Harold y Means, 2002), que se usa de forma amplia para describir elementos de datos estructurados.

Sistemas de procesamiento de datos

Como su nombre indica, estos sistemas se centran en datos, y las bases de datos con las que se relacionan son normalmente varios órdenes de magnitud más grandes que los propios Sistemas. Los sistemas de procesamiento de datos son sistemas de procesamiento por lotes en los que los datos son introducidos y extraídos por lotes a partir de un fichero o base de datos en lugar de ser introducidos y extraídos por un terminal de usuario. Estos sistemas seleccionan datos para el registro de entradas y, dependiendo del valor de los campos en los registros, realizan algunas acciones especificadas en el programa. Pueden volver a escribir el resultado del cálculo en la base de datos y formatea la entrada y la salida calculada para su impresión.

Los componentes de entrada, procesamiento y salida pueden descomponerse en una estructura de entrada-proceso-salida. Por ejemplo:



Las funciones en la parte izquierda del diagrama Lectura de registro de empleado, Lectura de datos a pagar mensualmente y Validación de datos de empleado introducen los datos para cada empleado y comprueban dichos datos.

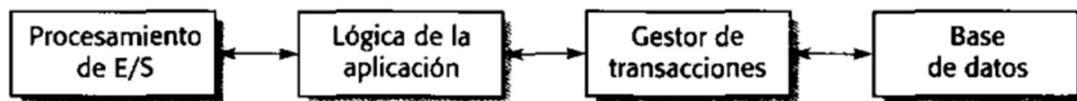
La función Calcular salario calcula el salario bruto para cada empleado y las deducciones a realizar sobre dicho salario.

Las funciones de salida escriben una serie de ficheros que contienen detalles de las deducciones realizadas y el salario a pagar.

Sistemas de procesamiento de transacciones

Los sistemas de procesamiento de transacciones se diseñan para procesar peticiones de usuario a fin de obtener información de una base de datos o peticiones para actualizar la base de datos. Técnicamente, una transacción de una base de datos es una secuencia de operaciones tratada como una única unidad (una unidad atómica). Todas las operaciones de una transacción tienen que

ser completadas antes de que los cambios en la base de datos sean permanentes. Esto significa que los fallos de operaciones dentro de la transacción no conducen a inconsistencias en la base de datos.



Primero un usuario realiza una petición al sistema a través de un componente de procesamiento de E/S. La petición se procesa por alguna lógica específica de la aplicación. Una transacción se crea y se envía a un gestor de transacciones, el cual normalmente está embebido en el sistema de gestión de base de datos. Después de que el gestor de transacciones haya asegurado que la transacción se ha realizado correctamente, envía una señal a la aplicación indicando que el procesamiento ha finalizado

La estructura entrada-procesa-salida que podemos observar en las aplicaciones de procesamiento de datos también se aplican a muchos sistemas de procesamiento de transacciones. Algunos de estos sistemas son versiones interactivas de los sistemas de procesamiento por lotes. Por ejemplo, por un lado, los bancos dan entrada fuera de línea a todas las transacciones de clientes, y después ejecutan por la noche estas transacciones en un lote utilizando la base de datos de cuentas. Esta aproximación se ha reemplazado en la mayoría de los casos por sistemas interactivos basados en transacciones que actualizan las cuentas en tiempo real.

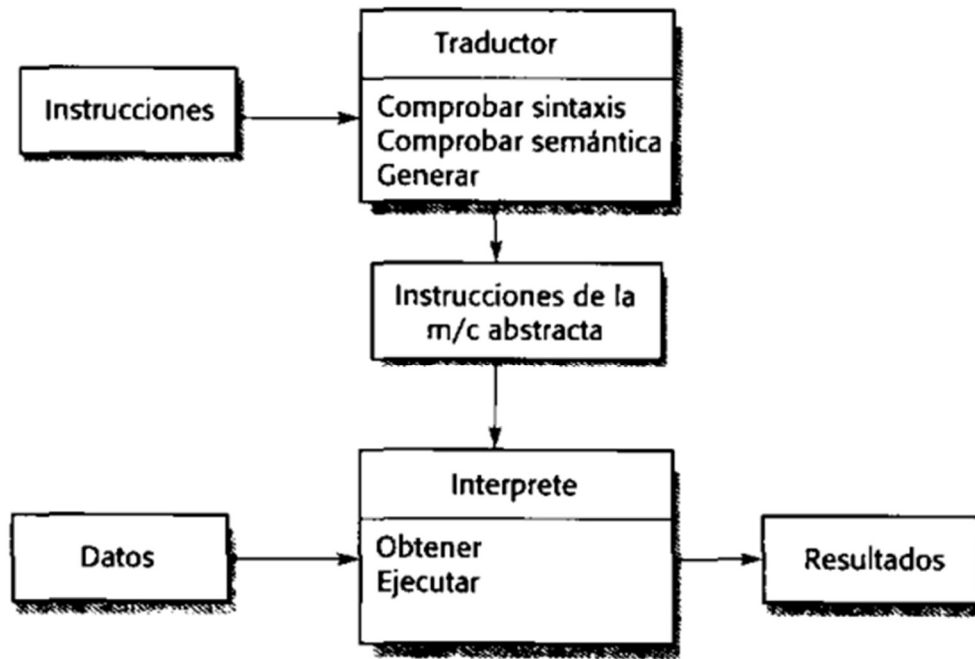
Sistemas de procesamiento de eventos

Todos nosotros usamos sistemas basados en eventos como éstos en nuestras propias computadoras -procesadores de texto, sistemas de presentaciones y juegos están todos conducidos por eventos desde la interfaz de usuario-. El sistema detecta e interpreta los eventos. Los eventos de interfaz de usuario representan comandos implícitos al sistema, que realiza alguna acción como respuesta a ese comando. Por ejemplo, si usted está utilizando un procesador de texto y pulsa dos veces el ratón sobre una palabra, el evento de doble pulsación significa «selecciona esa palabra»

Los sistemas de tiempo real, los cuales realizan acciones en «tiempo real» como respuesta a algunos estímulos externos, son también sistemas de procesamiento basados en eventos. Sin embargo, para sistemas de tiempo real, los eventos no son normalmente eventos de interfaz de usuario, sino eventos asociados con servidores o actuadores en el sistema. Debido a la necesidad de respuesta en tiempo real a eventos no predecibles, estos sistemas de tiempo real se organizan normalmente como un conjunto de procesos cooperativos.

Sistemas de procesamiento de lenguajes

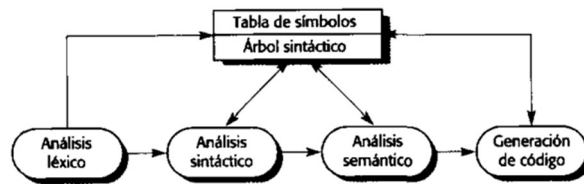
Los sistemas de procesamiento de lenguajes aceptan lenguaje natural o artificial como entrada y generan alguna otra representación de ese lenguaje como salida. En ingeniería del software, los sistemas de procesamiento de lenguajes más extensamente usados son los compiladores, que traducen un lenguaje de programación artificial de alto nivel a código máquina, pero otros sistemas de procesamiento de lenguajes traducen una descripción de datos en XML a comandos de consulta de una base de datos y sistemas de procesamiento de lenguaje natural que intentan traducir un lenguaje natural a otro.



Las instrucciones describen lo que tiene que realizarse y tienen que traducirse por un traductor a algún formato interno. Las instrucciones se corresponden con las instrucciones máquina para una máquina abstracta. A continuación, estas instrucciones son interpretadas por otro componente que obtiene las instrucciones para su ejecución y las ejecuta usando, si es necesario, datos del entorno. La salida del proceso es el resultado de interpretar las instrucciones sobre los datos de entrada. Por supuesto, para muchos compiladores el intérprete es una unidad hardware que procesa instrucciones máquina y la máquina abstracta es un procesador real. Sin embargo, para lenguajes como Java, el intérprete es un componente software.

Los traductores en un sistema de procesamiento de lenguajes tienen una arquitectura genérica (que incluye los siguientes componentes:

1. Un analizador léxico, que toma como entrada los símbolos del lenguaje y los convierte a un formato interno.
2. Una tabla de símbolos, que almacena información sobre los nombres de las entidades (variables, nombres de clases, nombres de objetos, etc.) usadas en el texto que se está traduciendo.
3. Un analizador sintáctico, que comprueba la sintaxis del lenguaje que se está traduciendo. Utiliza una gramática definida del lenguaje y construye un árbol sintáctico.
4. Un árbol sintáctico, que es una estructura interna que representa al programa que se está compilando.
5. Un analizador semántico, que utiliza información del árbol sintáctico y de la tabla de símbolos para comprobar la corrección semántica del texto en el lenguaje de entrada.
6. Un generador de código, que «recorre» el árbol sintáctico y genera código de máquina abstracta.



Arquitectura de los Sistemas Distribuidos

Modelos principales en los sistemas distribuidos

Cliente servidor

Monolitos

Microservicios

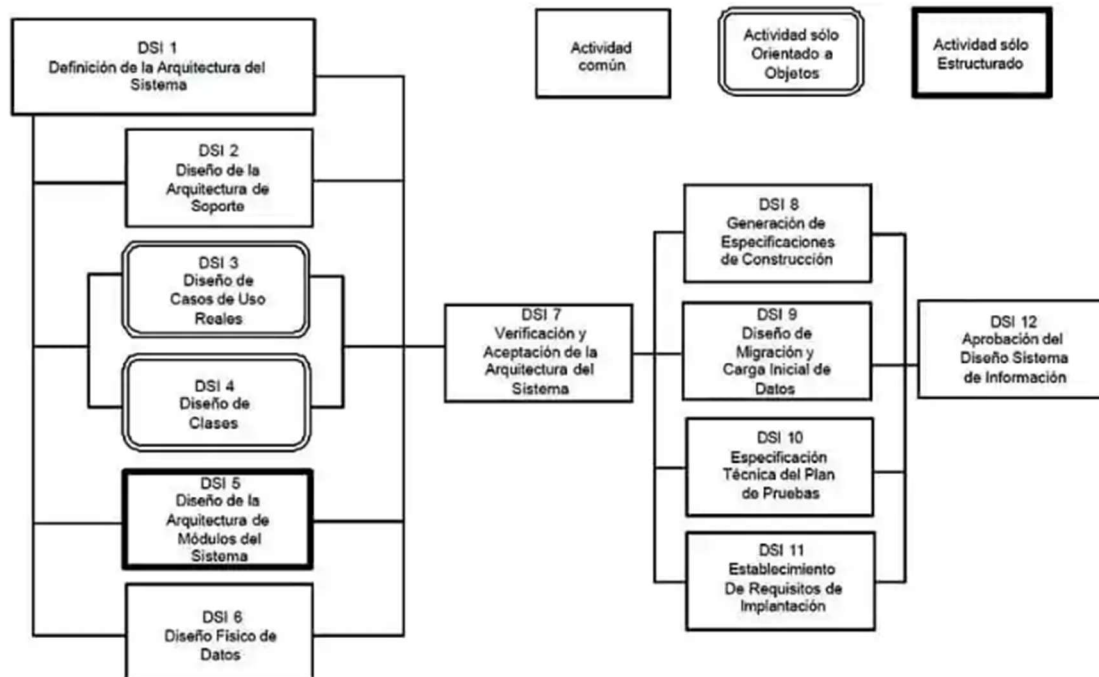
Proxy

Per to Per, P2P

Búsquedas distribuidas, enrutamiento

Tema 2. Diseño de Interfaz de Usuario.

Documentación del Diseño. Conceptos. Diseño vs. Modelo funcional. Abstracción, arquitectura, patrones, modularidad, ocultamiento de la información, independencia funcional, cohesión, acoplamiento, refinamiento.



1. Definición de la arquitectura del sistema

En esta tarea se describen los niveles de la arquitectura software, mediante la definición de las principales particiones físicas del sistema de información, representadas como nodos y comunicaciones entre nodos.

Se entiende por nodo cada partición física o parte significativa del sistema de información, con características propias de ejecución o función, e incluso de diseño y construcción.

2. Diseño de la arquitectura soporte

El diseño de los subsistemas de soporte, conceptualmente, es similar al diseño de los subsistemas específicos, aunque debe cumplir con unos objetivos claros de reutilización. De esta manera, se consigue simplificar y abstraer el diseño de los subsistemas específicos de la complejidad del entorno tecnológico, dotando al sistema de información de una mayor independencia de la infraestructura que le da soporte. Con este fin, se aconseja la consulta de los datos de otros proyectos existentes, disponible en el Histórico de Proyectos. Si esto no fuera suficiente, se puede contar en esta actividad con la participación de perfiles técnicos, con una visión global de la instalación.

3. Diseño de casos de usos reales

Esta actividad, que se realiza solo en el caso de **Diseño Orientado a Objetos**, tiene como propósito especificar el comportamiento del sistema de información para un caso de uso, mediante objetos o

subsistemas de diseño que interactúan, y determinar las operaciones de las clases e interfaces de los distintos subsistemas de diseño.

Para ello, una vez identificadas las clases participantes dentro de un caso de uso, es necesario completar los escenarios que se recogen del análisis, incluyendo las clases de diseño que correspondan y teniendo en cuenta las restricciones del entorno tecnológico, esto es, detalles relacionados con la implementación del sistema. Es necesario analizar los comportamientos de excepción para dichos escenarios. Algunos de ellos pueden haber sido identificados en el proceso de análisis, aunque no se resuelven hasta este momento. Dichas excepciones se añadirán al catálogo de excepciones para facilitar las pruebas.

4. Diseño de Clases

El propósito de esta actividad, que se realiza sólo en el caso de ***Diseño Orientado a Objetos***, es transformar el modelo de clases lógico, que proviene del análisis, en un modelo de clases de diseño. Dicho modelo recoge la especificación detallada de cada una de las clases, es decir, sus atributos, operaciones, métodos, y el diseño preciso de las relaciones establecidas entre ellas, bien sean de agregación, asociación o jerarquía. Para llevar a cabo todos estos puntos, se tienen en cuenta las decisiones tomadas sobre el entorno tecnológico y el entorno de desarrollo elegido para la implementación.

Se identifican las clases de diseño, que denominamos clases adicionales, en función del estudio de los escenarios de los casos de uso, que se está realizando en paralelo en la actividad ***Diseño de Casos de Uso Reales***, y aplicando los mecanismos genéricos de diseño que se consideren convenientes por el tipo de especificaciones tecnológicas y de desarrollo. Entre ellas se encuentran clases abstractas, que integran características comunes con el objetivo de especializarlas en clases derivadas. Se diseñan las clases de interfaz de usuario, que provienen del análisis. Como consecuencia del estudio de los escenarios secundarios que se está realizando, pueden aparecer nuevas clases de interfaz.

5. Diseño de la arquitectura de módulos de sistemas

El objetivo de esta actividad, que sólo se realiza en el caso de ***Diseño Estructurado***, es definir los módulos del sistema de información, y la manera en que van a interactuar unos con otros, intentando que cada módulo trate total o parcialmente un proceso específico y tenga una interfaz sencilla.

Durante el diseño de los módulos, se pueden identificar características o comportamientos comunes relacionados con accesos a las bases de datos o ficheros, lógica de tratamiento, llamadas a otros módulos, gestión de errores, etc. que determinen la necesidad de realizar su implementación como subsistemas de soporte.

6. Diseño Físico de Datos

Se define la estructura física de datos que utilizará el sistema, a partir del modelo lógico de datos normalizado o modelo de clases, de manera que teniendo presentes las características específicas del sistema de gestión de datos concreto a utilizar, los requisitos establecidos para el sistema de información, y las particularidades del entorno tecnológico, se consiga una mayor eficiencia en el tratamiento de los datos.

También se analizan los caminos de acceso a los datos utilizados por cada módulo/clase del sistema en consultas y actualizaciones, con el fin de mejorar los tiempos de respuesta y optimizar los recursos de máquina.

7. Verificaciones y aceptación de la arquitectura del sistema

El objetivo de esta actividad es garantizar la calidad de las especificaciones del diseño del sistema de información y la viabilidad del mismo, como paso previo a la generación de las especificaciones de construcción.

Para cumplir dicho objetivo, se llevan a cabo las siguientes acciones:

- Verificación de la calidad técnica de cada modelo o especificación
- Aseguramiento de la coherencia entre los distintos modelos
- Aceptación del diseño de la arquitectura por parte de Explotación y Sistemas.

8. Generalización de especializaciones de construcción

En esta actividad se generan las especificaciones para la construcción del sistema de información, a partir del diseño detallado.

Estas especificaciones definen la construcción del sistema de información a partir de las unidades básicas de construcción (en adelante, componentes), entendiendo como tales unidades independientes y coherentes de construcción y ejecución, que se corresponden con un empaquetamiento físico de los elementos del diseño de detalle, como pueden ser módulos, clases o especificaciones de interfaz.

La división del sistema de información en subsistemas de diseño proporciona, por continuidad, una primera división en subsistemas de construcción, definiendo para cada uno de ellos los componentes que lo integran. Si se considera necesario, un subsistema de diseño se podrá dividir a su vez en sucesivos niveles para mayor claridad de las especificaciones de construcción.

Las dependencias entre subsistemas de diseño proporcionan información para establecer las dependencias entre los subsistemas de construcción y, por lo tanto, definir el orden o secuencia que se debe seguir en la construcción y en la realización de las pruebas.

9. Diseño de migración y carga inicial de Datos

Sólo se lleva a cabo cuando es necesaria una carga inicial de información, o una migración de datos de otros sistemas, cuyo alcance y estrategia a seguir se habrá establecido previamente.

Para ello, se toma como referencia el plan de migración y carga inicial de datos, que recoge las estructuras físicas de datos del sistema o sistemas origen implicadas en la conversión, la prioridad en las cargas y secuencia a seguir, las necesidades previas de depuración de la información, así como los requisitos necesarios para garantizar la correcta implementación de los procedimientos de migración sin comprometer el funcionamiento de los sistemas actuales.

10. Especificación técnica de plan de Prueba

En esta actividad se realiza la especificación de detalle del plan de pruebas del sistema de información para cada uno de los niveles de prueba establecidos en el proceso **Análisis del Sistema de Información**.

Para ello se toma como referencia el plan de pruebas, que recoge los objetivos de la prueba de un sistema, establece y coordina una estrategia de trabajo, y provee del marco adecuado para planificar paso a paso las actividades de prueba

11. Establecimiento de requisitos de implementación

En esta actividad se completa el catálogo de requisitos con aquéllos relacionados con la documentación que el usuario requiere para operar con el nuevo sistema, y los relativos a la propia implantación del sistema en el entorno de operación.

12. Aprobación del Diseño Sistemas de Información

Acoplamiento

El acoplamiento entre módulos clasifica el grado de independencia entre pares de módulos de un DE. El objetivo es minimizar el acoplamiento, es decir, maximizar la independencia entre módulos. A pesar de que el acoplamiento, es un criterio que clasifica características de una invocación

Un bajo acoplamiento indica un sistema bien particionado y puede obtenerse de tres maneras:

- Eliminando relaciones innecesarias: Por ejemplo, un módulo puede recibir algunos datos, innecesarios para él, porque debe enviarlos para un módulo subordinado.
- Reduciendo el número de relaciones necesarias: Cuantas menos conexiones existan entre módulos, menor será la posibilidad del efecto en cadena (un error en un módulo aparece como síntoma en otro).
- Debilitando la dependencia de las relaciones necesarias: Ningún módulo se tiene que preocupar por los detalles internos de implementación de cualquier otro. Lo único que tiene que conocer un módulo debe ser su función y las cuplas de entrada y salida (cajas negras).

Cohesión

Otro medio para evaluar la partición en módulos (además del acoplamiento) es observar cómo las actividades de un módulo están relacionadas unas con otras; este es el criterio de cohesión. Generalmente el tipo de cohesión de un módulo determina el nivel de acoplamiento que tendrá con otros módulos del sistema.

Cohesión es la medida de intensidad de asociación funcional de los elementos de un módulo. Por elemento debemos entender una instrucción, o un grupo de instrucciones o una llamada a otro módulo, o un conjunto de procedimientos o funciones empaquetados en el mismo módulo. El objetivo del diseño estructurado es obtener módulos altamente cohesivos, cuyos elementos estén fuerte y genuinamente relacionados unos con otros. Por otro lado, los elementos de un módulo no deberían estar fuertemente relacionados con elementos de otros módulos, porque eso llevaría a un fuerte acoplamiento entre ellos.

Descomposición

La descomposición es la separación de una función contenida en un módulo, para un nuevo módulo. Puede ser hecha por cualquiera de las siguientes razones.

Reducir el tamaño del módulo

La descomposición es una manera eficiente de trabajar con módulos grandes. Un buen tamaño para un módulo es alrededor de media página (30 líneas). Ciertamente, toda codificación de un módulo debería ser visible en una página (60 líneas).

Otros criterios para determinar cuándo es conveniente terminar de realizar la descomposición, son los siguientes:

- **Funcionalidad:** Terminar de realizar la descomposición cuando no se pueda encontrar una función bien definida. No empaquetar líneas de código dispersas, de otros módulos, porque probablemente juntas podrán formar módulos con mala cohesión.
- **Complejidad de Interfaz:** Terminar de realizar la descomposición cuando la interfaz de un módulo es tan compleja como el propio módulo. Un módulo de mil líneas es muy confuso, más mil módulos de una línea son aún más confusos

Hacer el sistema más claro

En una descomposición no se debe preocupar por conceptos de programación. Si una subfunción, presentada como un módulo separado permite una mejor comprensión del diseño, puede ser subdividida, aun cuando, en una implementación, el código del módulo sea programado dentro del módulo jefe

Minimizar la duplicación de código

Cuando se reconoce una función que puede ser reutilizada en otras partes del DE, lo más conveniente es convertirla en un módulo separado. Así, se puede localizar más fácilmente las funciones ya identificadas y evitar la duplicación del mismo código en el interior de otro módulo.

Separar el trabajo de la ‘administración’

El resultado de este tipo de organización es un sistema en el cual los módulos en los niveles medio y alto de un DE son fáciles de implementar, porque ellos obtienen el trabajo hecho por la manipulación de los módulos de los niveles inferiores.

La separación del trabajo de la administración mejora la mantenibilidad del diseño. Una alteración en un sistema es: un cambio de control o un cambio de trabajo, pero raramente ambos.

Crear módulos más generales

Otra ventaja de la descomposición es que, frecuentemente, se pueden reconocer módulos más generales y, así, más útiles y reutilizables en el mismo sistema y, además, pueden ser generadas bibliotecas de módulos reutilizables en otros sistemas.

Documentación

La documentación del diseño es vital para garantizar su legibilidad y correcto entendimiento en la etapa de construcción o codificación, además de permitir detectar errores de consistencia. Debe entenderse que el diseño debe ser perfectible por personas ajenas a quienes lo construyeron, por lo que la documentación no puede faltar

Para documentar el diseño se tienen que documentar todos los elementos que aparecen en los diagramas de flujos de datos y diagramas de estructuras, esto es: Terminadores, almacenes de datos, flujos de datos y procesos.

Tema 3. El modelo de datos relacional.

El modelo de Datos desde la especificación del diseño. Refinamiento del modelo conceptual de datos. Modelo lógico de datos. Fases cualitativas, cuantitativa y de optimización. Normalización. Diseño objeto relacional. Consideraciones de la implementación. Documentación.

A través de la definición del Modelo Lógico y Físico del sistema, se conformará el Modelo de Datos del mismo en sus distintas perspectivas.

Un buen modelo de datos no se conseguirá de una sola vez, será necesario realizar sucesivos refinamientos para alcanzar así niveles de calidad esperados.

Aplicar los Niveles de Normalización, hasta el nivel más conveniente para el diseño y performance deseado.

No siempre es recomendable el máximo nivel.

No perder de vista el Objetivo y el alcance del Sistema.

La determinación de que se debe cubrir determina que almacenar.

Factores críticos:

Trabajar interactivamente con los usuarios

Seguir una metodología (pasos para su desarrollo)

Considerar tanto la estructura de la información como la integridad de esta. (parte del principio del dominio de la información)

Utilizar diagramas para representar el modelo de datos lógico (principio de modelado)

Construir un diccionario de datos

Beneficios:

Un correcto diseño de la base de datos.

Ayuda a valorar cuál es la tecnología óptima para desarrollar la base de datos del sistema. Permite reconocer como van a afectar posibles cambios en el futuro.

Permite a los usuarios comprender cuáles serán sus datos en el sistema final sin tener este implementado.

Favorece una visión del sistema y de cuáles son las necesidades reales de información.

El modelo de datos facilita la migración de una base de datos a otra.

Flexibilidad.

Evaluar considerar campos parametrizables.

Mantenible.

El posible cambio en la estructura, debería ser lo menos costoso posible.

Portable.

Analizar la conexión con posibles esquemas de datos relacionados y su interacción.

Tipos de entidades

Regulares. Son las entidades normales que tienen existencia por sí mismas sin depender de otras.

Débiles. Su existencia depende de otras. Las entidades débiles pueden representarse con doble línea

Elementos del modelado

Relaciones

Representan asociaciones entre entidades. Es el elemento del modelo que permite relacionar en sí los datos del modelo. Por ejemplo, en el caso de que tengamos una entidad personas y otra entidad trabajos. Ambas se realizan ya que las personas trabajan y los trabajos son realizados por personas.

Atributos

Describen propiedades de las entidades y las relaciones. En este modelo se representan con un círculo, dentro del cual se coloca el nombre del atributo

Cardinalidad

Indica el número de relaciones en las que una entidad puede aparecer. Se anota en términos de: *cardinalidad mínima*, indica el número mínimo de asociaciones; *cardinalidad máxima*, indica el número máximo de relaciones en las que puede aparecer cada ejemplar de la entidad

Regla de negocio

Proporcionan: Integridad, Consistencia y corrección

Reglas que afectan a las operaciones de borrar, insertar

En algunos casos la definición de la cardinalidad ya implica una obligatoriedad en el comportamiento de la relación.

Independencia lógica/física

Independencia física de los datos. Aunque el esquema físico cambie, el esquema conceptual no debe verse afectado. En la práctica esto significa que, aunque se añadan o cambien discos u otro hardware, o se modifique el sistema operativo u otros cambios relacionados con la física de la base de datos, el esquema conceptual permanece invariable

Independencia lógica de los datos. Significa que, aunque se modifique el esquema conceptual, la vista que poseen las aplicaciones (los esquemas externos) no serán afectados.

Diseño de Datos

Objetivo: generar un modelo de datos que permita almacenar los datos sin innecesaria redundancia y que permita un fácil acceso a los mismos.

Problemas •

Información Repetida

Inhabilidad de Representación

Pérdida de Información

La mayoría de estos problemas pueden ser resueltos en el modelo lógico de datos (Entidad-Relación).

Normalización

Es un proceso reversible mediante el cual los datos de un modelo de datos se ponen en la forma más simple e independiente.

Permite que, si la estructura física para los datos que se implemente esta normalizada, siempre será posible satisfacer cualquier requerimiento sobre los datos

Primera Forma Normal

Una relación R se encuentra en 1FN si y solo si por cada renglón columna contiene **valores atómicos**.

Segunda Forma Normal

Todos los atributos que no forman parte de la **Clave Principal** tienen dependencia funcional completa de ella. Dicho de otra forma: No debe haber dependencias parciales de ninguna clave candidata en ninguna columna no clave.

Tercera Forma Normal

Ocurre cuando una tabla está en 2FN y además ningún atributo que no sea clave depende transitivamente de las claves de la tabla. Es decir no ocurre cuando algún atributo depende funcionalmente de atributos que no son clave.

Eliminación de redundancia

Dos entidades tienen redundancia de estructura si sus atributos son iguales (o un % dado), pero no hay una ocurrencia del mismo sujeto en cada entidad.

Dos entidades tienen redundancia de datos si una misma ocurrencia puede tener los mismos datos en ocurrencias distintas de las entidades

Desnormalización

El proceso de desnormalización suele ser requerido en requerimientos de gestión de informes por ejemplo cantidad de alumnos, del departamento de Informática, en el segundo llamado de examen final. El fin es estadístico y no tener una información estandarizada o normalizada.

Tema 4. Diseño Orientado a Objetos.

De los requisitos al diseño. Proceso del Diseño Orientado a Objetos. Contexto del Sistema y Modelos de Utilización. Objetos Concurrentes. Diseño de la Arquitectura. Especificación de la Interfaz de Objetos. La documentación del Diseño y la importancia de la trazabilidad. Evolución del diseño. Diseño de Software de Tiempo Real.

Un sistema orientado a objetos está compuesto de objetos que interactúan, los cuales mantienen ellos mismos su estado local y proveen operaciones sobre su estado. El proceso de diseño orientado a objetos comprende el diseño de clases de objetos y las relaciones entre estas clases. Las clases definen los objetos del sistema y sus interacciones. Cuando el diseño se implementa como un programa ejecutable, los objetos requeridos se crean dinámicamente utilizando las definiciones de las clases.

El diseño orientado a objetos es parte del desarrollo orientado a objetos en el que se utiliza una estrategia orientada a objetos en el proceso de desarrollo:

El análisis orientado a objetos comprende el desarrollo de un modelo orientado a objetos del dominio de aplicación. Los objetos identificados reflejan las entidades y operaciones que se asocian con el problema a resolver

El diseño orientado a objetos comprende el desarrollo de un modelo orientado a objetos de un sistema software para implementar los requerimientos identificados. Los objetos en un diseño orientado a objetos están relacionados con la solución del problema por resolver.

La programación orientada a objetos se refiere a implementar el diseño de software utilizando un lenguaje de programación orientado a objetos, como Java.

La transición entre estas etapas de desarrollo se lleva a cabo, idealmente, sin problemas, utilizando notaciones compatibles entre las etapas. Pasar a la siguiente etapa implica refinar la etapa previa agregando algún detalle a las clases existentes y crear nuevas clases con el fin de proveer nuevas funcionalidades. Puesto que la información se oculta dentro de los objetos, las decisiones del diseño detallado de la representación de los datos se pueden retrasar hasta que el sistema se implemente.

Objetos y clases

Un objeto es una entidad que tiene un estado y un conjunto de operaciones definidas que operan sobre ese estado. El estado se representa como un conjunto de atributos del objeto. Las operaciones asociadas al objeto proveen servicios a otros objetos que solicitan estos servicios cuando se requiere llevar a cabo algún cálculo.

Los objetos se crean conforme a una definición de clases de objetos. Una definición de clases sirve como una plantilla para crear objetos. Ésta incluye las declaraciones de todos los atributos y operaciones asociados con un objeto de esa clase.

Objetos concurrentes

Existen dos clases de implementación de objetos concurrentes:

Servidores en los cuales el objeto se implementa como un proceso paralelo con métodos que corresponden a las operaciones definidas de los objetos. Los métodos inician su actividad en respuesta a un mensaje externo y se ejecutan en paralelo con los métodos

asociados a otros objetos_ Cuando han completado su operación, el objeto se suspende por sí mismo y espera por las peticiones adicionales de los servicios.

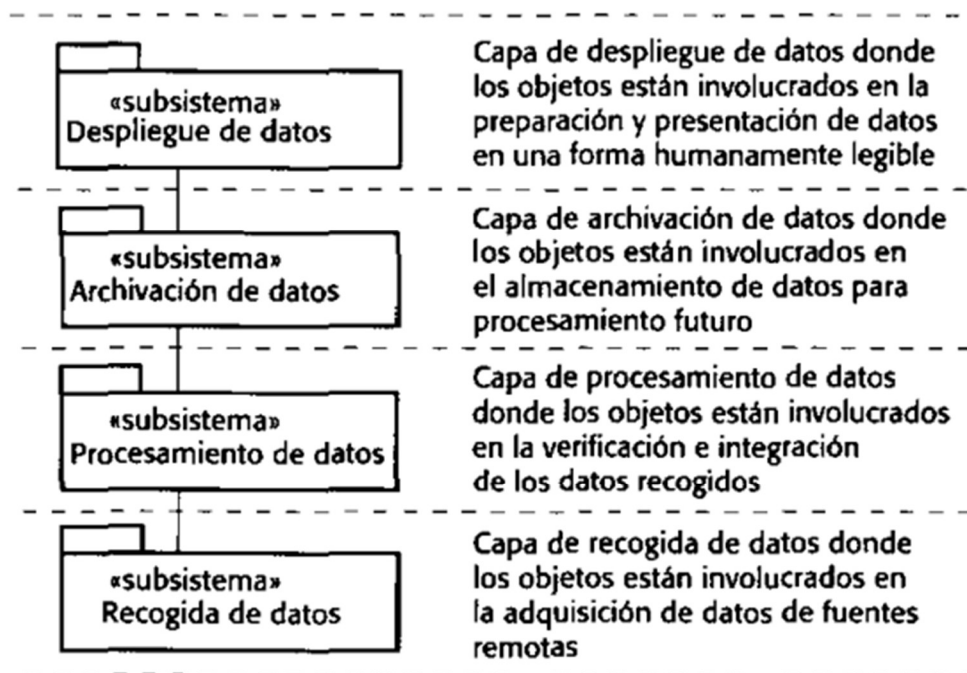
Objetos activos en los cuales el estado de los objetos cambia debido a la ejecución de operaciones internas. El proceso que representa al objeto ejecuta continuamente estas operaciones, por lo que no se suspende por sí solo.

Un proceso de diseño orientado a objetos

El proceso general que aquí se utiliza para el diseño orientado a objetos tiene varias etapas:

1. Comprender y definir el contexto y los modos de utilización del sistema.
2. Diseñar la arquitectura del sistema.
3. Identificar los objetos principales en el sistema.
4. Desarrollar los modelos de diseño.
5. Especificar las interfaces de los objetos.

No se muestra como un diagrama de proceso simple, puesto que esto implicaría la existencia de actividades detalladas llevadas a cabo en secuencia. De hecho, todas las actividades anteriores se pueden ver como actividades entrelazadas que influyen entre sí. Los objetos se identifican y las interfaces se especifican completa o parcialmente en el momento de definir la arquitectura del sistema. En el momento de elaborar los modelos de objetos, estas definiciones de los objetos se refinan y pueden implicar un cambio en la arquitectura del sistema.



En la figura, se observan las diferentes capas y se incluye el nombre de la capa en un símbolo de representación de paquetes en UML que se ha denotado como un subsistema. Un paquete en UML representa una colección de objetos y otros paquetes. Aquí se ha utilizado para mostrar que cada capa incluye otros componentes.

Contexto del sistema y modelos de utilización

La primera etapa en el proceso de diseño de software es comprender las relaciones entre el software que se está diseñando y el entorno externo. Comprender esto ayuda a decidir cómo suministrar la funcionalidad requerida al sistema y cómo estructurar éste para que se comunique efectivamente con su entorno. El contexto del sistema y el modelo de utilización del sistema representan dos modelos complementarios entre un sistema y su entorno:

1. El contexto del sistema es un modelo estático que describe a los otros sistemas de su entorno.
2. El modelo de utilización del sistema es un modelo dinámico que describe cómo el sistema interactúa con su entorno

Cuando se modelan las interacciones de un sistema con su entorno, se debe utilizar un enfoque abstracto que no incluya mucho detalle de esas interacciones. El enfoque que se propone en UML es desarrollar un modelo de casos de uso donde cada caso de uso representa una interacción con el sistema. En los modelos de caso de uso (descritos también en el Capítulo 7), cada interacción posible se enuncia en una elipse y la entidad externa implicada en la interacción se representa mediante una figura estilizada. En el caso del sistema de la estación meteorológica esta entidad externa no es un humano sino el sistema de procesamiento para los datos meteorológicos.

Contexto del sistema y modelos de utilización

La primera etapa en el proceso de diseño de software es comprender las relaciones entre el software que se está diseñando y el entorno externo. Comprender esto ayuda a decidir cómo suministrar la funcionalidad requerida al sistema y cómo estructurar éste para que se comunique efectivamente con su entorno. El contexto del sistema y el modelo de utilización del sistema representan dos modelos complementarios entre un sistema y su entorno:

1. El contexto del sistema es un modelo estático que describe a los otros sistemas de su entorno.
2. El modelo de utilización del sistema es un modelo dinámico que describe cómo el sistema interactúa con su entorno.

Diseño de la arquitectura

Una vez que se han definido las interacciones entre el sistema de software que se está diseñando y el entorno del sistema, se puede utilizar esta información como base para diseñar la arquitectura del sistema. Por supuesto, es necesario combinar esto con el conocimiento general de los principios de diseño arquitectónico y con el conocimiento más detallado del dominio

En general, se debe tratar de descomponer un sistema de tal forma que las arquitecturas sean lo más sencillas posible. Una regla que siempre funciona es que no debe haber más de siete entidades fundamentales en un modelo arquitectónico. Cada una de estas entidades se puede describir independientemente, pero, por supuesto, se debe dejar al descubierto la estructura de las entidades.

Identificación de objetos

En esta etapa del proceso de diseño ya tuvo que haber formulado algunas ideas de los objetos esenciales del sistema que se está diseñando. Esto refleja un principio general, el cual señala que

los objetos tienden a emerger durante el proceso de diseño. Sin embargo, por lo general, hay que buscar y documentar otros objetos que pudieran ser relevantes.

De forma inevitable, se tienen que refinar las clases identificadas de forma inicial y volver a esta etapa del proceso conforme se vaya obteniendo una comprensión más profunda del diseño. Se han hecho varias propuestas de cómo identificar las clases:

Utilizar un análisis gramatical de la descripción en lenguaje natural del sistema. Los objetos y los atributos son sustantivos; las operaciones o servicios son verbos.

Utilizar entidades tangibles (cosas) en el dominio de aplicación como aviones, papeles como administrador, eventos como una petición, interacciones como reuniones, ubicaciones como oficinas, unidades organizacionales como compañías, etcétera.

Utilizar un enfoque de comportamiento en el cual el diseñador primero comprende el comportamiento total del sistema. Los diversos comportamientos se asignan a distintas partes del sistema para así comprender quién inicia y participa en estos comportamientos.

Estos enfoques ayudan en el inicio de la identificación de objetos. En la práctica, diversas fuentes de conocimiento se utilizan para descubrir objetos y clases. Los objetos y las operaciones que se identifican inicialmente de la descripción informal del sistema pueden ser un punto de partida para el diseño.

Los modelos de diseño muestran los objetos o clases en un sistema y, donde sea apropiado, los diferentes tipos de relaciones entre estas entidades. Los modelos de diseño son esencialmente el diseño mismo. Son el puente entre los requerimientos y la implementación del sistema. Esto significa que existen requerimientos en conflicto en estos modelos. Tienen que ser abstractos con el fin de que el detalle innecesario no oculte las relaciones entre ellos y los requerimientos del sistema.

Existen dos tipos de modelos de diseño para describir un diseño orientado a objetos:

Modelos estáticos que describen la estructura estática del sistema en términos de las clases del sistema y sus relaciones. Las relaciones importantes que se documentan en esta etapa son de generalización, utiliza/utilizado-por y de composición.

Modelos dinámicos que describen la estructura dinámica del sistema y que muestran las interacciones entre los objetos del sistema (no entre las clases). Las interacciones que se documentan incluyen la secuencia de servicios solicitados por los objetos y la forma en que el estado del sistema se relaciona con estas interacciones de objetos.

Un modelo de subsistemas es un modelo estático útil que nos muestra cómo puede estar organizado el diseño de forma lógica agrupando objetos. Los paquetes UML contienen las construcciones de encapsulación y no se reflejan directamente sobre las entidades del sistema que se implementa.

Los modelos de secuencia son modelos dinámicos que documentan, para cada modo de interacción, la secuencia de interacciones que tienen lugar entre los objetos.

Los diagramas de secuencia se utilizan para modelar el comportamiento combinado de un grupo de objetos, pero también son útiles para resumir el comportamiento de un solo objeto como respuesta a los diversos mensajes que puede procesar. Para hacer esto, se utiliza un modelo de máquinas de estado que muestra como la instancia del objeto cambia de estado dependiendo de los mensajes que reciba.

Diagramas. UML

DIAGRAMAS DE ESTRUCTURA	DIAGRAMAS DE COMPORTAMIENTO	Diagramas de Interacción
Diagrama de Clases	Diagrama de Actividad Diagrama de Caso de Uso	Diagrama de Secuencia
Diagrama de Componentes	Diagrama de Caso de Uso	Diagrama de Comunicación o Colaboración
Diagrama de Objetos	Diagrama de Estados	
Diagrama de Despliegue o Distribución		
Diagrama de Paquetes		

Especificación de la interfaz de los objetos

Es necesario identificar las interfaces para que los objetos y otros componentes se puedan diseñar en paralelo. Una vez que la interfaz se ha especificado, los desarrolladores de otros objetos pueden suponer que la interfaz será implementada.

El diseño de interfaces de objetos comprende la especificación del detalle de la interfaz para un objeto o un grupo de objetos. Esto significa definir las formas y semántica de los servicios proporcionados por los objetos o por un grupo de objetos. Las interfaces se especifican en UML utilizando la misma notación que en los diagramas de clases.

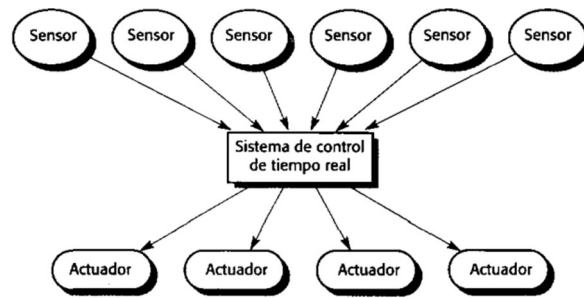
Evolución del diseño

Una ventaja importante de un enfoque orientado a objetos para el diseño es que simplifica el problema de hacer cambios a dicho diseño. La razón de esto es que la representación del estado del objeto no influye en el diseño. Cambiar los detalles internos de un objeto no afecta a ningún otro objeto del sistema. Más aún, debido a que los objetos están débilmente acoplados, por lo regular es sencillo introducir nuevos objetos sin efectos importantes en el resto del sistema.

Diseño de software de tiempo real

Un sistema de tiempo real es un sistema software cuyo correcto funcionamiento depende de los resultados producidos por el mismo y del instante de tiempo en el que se producen estos resultados. Un sistema de tiempo real blando (soft) es un sistema cuyo funcionamiento se degrada si los resultados no se producen de acuerdo con los requerimientos temporales especificados. Un sistema de tiempo real duro (hard) es un sistema cuyo funcionamiento es incorrecto si los resultados no se producen de acuerdo con la especificación temporal.

Una forma de ver un sistema de tiempo real es como un sistema de estímulo/respuesta. Dado un determinado estímulo de entrada, el sistema debe producir la correspondiente salida. Se puede, por lo tanto, definir el comportamiento de un sistema de tiempo real haciendo una lista de los estímulos recibidos por el sistema, las respuestas asociadas y el tiempo en el que dichas respuestas deben producirse.



Las arquitecturas de aplicaciones de tiempo real son instancias de la arquitectura conducida por eventos en la cual el estímulo, directa o indirectamente, provoca la generación de eventos.

Los lenguajes de programación desarrollados para sistemas de tiempo real tienen que incluir facilidades para acceder al hardware del sistema, y debería ser posible predecir la duración de operaciones particulares realizadas en estos lenguajes.

Los procesos en un sistema de tiempo real tienen que coordinarse. Los mecanismos de coordinación de procesos aseguran la exclusión mutua de recursos compartidos. Cuando un proceso modifica un recurso compartido, al resto de los procesos no se les debería permitir cambiar ese recurso.

Una vez que se ha elegido la plataforma de ejecución para el sistema, se ha diseñado una arquitectura para el proceso y se ha decidido una política de planificación (scheduling). puede necesitarse comprobar que el sistema satisface sus requerimientos temporales.

El análisis temporal para sistemas de tiempo real es difícil. Debido a que los estímulos aperiódicos son impredecibles, los diseñadores tienen que hacer suposiciones sobre la probabilidad de ocurrencia de estos estímulos (y por lo tanto del servicio requerido) en un instante de tiempo concreto.

Debido a que los sistemas de tiempo real deben satisfacer sus restricciones temporales. es posible que no se pueda usar el desarrollo orientado a objetos para sistemas de tiempo real duros. El desarrollo orientado a objetos implica la ocultación de representaciones de datos y el acceso a los valores de los atributos a través de operaciones definidas en el objeto.

Los modelos de máquina de estados son una buena aproximación independiente del lenguaje de representar el diseño de un sistema de tiempo real y. por lo tanto, son una parte integral de los métodos de diseño de sistemas de tiempo real

Un sistema operativo de tiempo real gestiona los procesos y asignación de recursos en un sistema de tiempo real. Lanza y detiene los procesos para que los estímulos puedan ser manejados y asigna memoria y recursos del procesador.

Tema 5. Patrones de Diseño.

Patrones de Diseño. Aplicación de Graps. Conceptos Fundamentales. Características principales de los distintos Patrones. Adaptador. Factoría. Singleton. Estrategia. Composite. Fachada. Observador/Publicar/Suscribir/Modelos de Delegación de Eventos. Ventajas sobre su aplicación.

Un patrón de diseño se caracteriza como “una regla de tres partes que expresa una relación entre cierto contexto, un problema y una solución”. Para el diseño de software, el contexto permite al lector entender el ambiente en el que reside el problema y qué solución sería apropiada en dicho ambiente. Un conjunto de requerimientos, incluidas limitaciones y restricciones, actúan como sistema de fuerzas que influyen en la manera en la que puede interpretarse el problema en este contexto y en cómo podría aplicarse con eficacia la solución.

El conjunto de requerimientos, incluidas limitaciones y restricciones, son los que hace que un diseñador elija una solución específica. El objetivo es proporcionar la que satisfaga mejor al sistema de fuerzas, aun cuando éstas sean contradictorias. Por último, toda solución tiene consecuencias que afectan otros aspectos del software y que se vuelven parte del sistema de fuerzas de otros problemas por resolver en el sistema mayor.

Aplicación de Graps

Los patrones GRASP constituyen un apoyo para la enseñanza que ayuda a uno a entender el diseño de objetos esencial, y aplica el razonamiento para el diseño de una forma sistemática, racional y explicable. Este enfoque para la comprensión y utilización de los principios de diseño se basa en los patrones de asignación de responsabilidades.

GRASP es un acrónimo de **General Responsibility Assignment Software Patterns** (patrones generales de software para asignar responsabilidades) . El nombre se eligió para sugerir la importancia de aprehender (grasping en inglés) estos principios para diseñar con éxito el software orientado a objetos.

Cómo aplicar los patrones GRASP

Patrones GRASP:

Experto en Información.

Asignar una responsabilidad al experto en información, *la clase que tiene la información necesaria para realizar la responsabilidad.*

Clase Venta va saber el total, VentaDetalle conoce su subtotal y Producto sabe su precio. Es decir, a Venta le vamos a dar de saber el total de la venta `getVentaTotal()`, `VentaDetalle getSubtotalItem()`, y `Producto getPrecio()`.

Creador.

La creación de instancias es una de las actividades más comunes en un sistema orientado a objetos. En consecuencia, es útil contar con un principio general para la asignación de las responsabilidades de creación.

Asignar a la clase B la responsabilidad de crear una instancia de clase A si se cumple uno o más de los casos siguientes:

B agrega objetos de A.

B contiene objetos de A.

B registra instancias de objetos de A.

B utiliza más estrechamente objetos de A.

B es un creador de los objetos A.

Un CarritoCompra contiene muchos Productos. Es por eso que le patrón Creador sugiere que CarritoCompra es la clase idónea para asumir la responsabilidad de crear instancia de Productos.

Alta Cohesión.

Asignar una responsabilidad de manera que la cohesión permanezca alta. Esto mantiene un bajo acoplamiento. De lo contrario, vamos a tener clases que son difícil de entender, reutilizar y mantener.

Una clase con alta cohesión tiene un numero relativamente pequeño de métodos con funcionalidades altamente relacionadas.

Bajo Acoplamiento.

El acoplamiento es una medida de la fuerza con que un elemento está conectado a, tiene conocimiento de, confía en, otros elementos. Un elemento con bajo (o débil) acoplamiento no depende de demasiados otros elementos; "demasiados" depende del contexto, pero se estudiará. Estos elementos pueden ser clases, subsistemas, sistemas, etcétera.

Las clases son independientes. Si cambias el código interno de la Clase A, la Clase B no debería romperse. Esto se logra usando interfaces y abstracciones

Si el Bajo Acoplamiento se lleva al extremo, producirá un diseño pobre porque dará lugar a unos pocos objetos inconexos, saturados, y con actividad compleja que hacen todo el trabajo, con muchos objetos muy pasivos, sin acoplamiento que actúan como simples repositorios de datos.

Controlador.

Asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase que representa una de las siguientes opciones:

Representa el sistema global, dispositivo o subsistema (controlador de fachada).

Representa un escenario de caso de uso en el que tiene lugar el evento del sistema,

Caracteriza un patrón de diseño eficaz del modo siguiente:

Resuelve un problema: los patrones entrañan soluciones, no sólo principios o estrategias abstractas.

Es un concepto probado: los patrones incluyen soluciones con un historial, no teorías o especulaciones.

La solución no es obvia: muchas técnicas de solución de problemas (como los paradigmas o métodos de diseño de software) tratan de obtener soluciones a partir de sus principios originales. Los mejores patrones generan indirectamente una solución a un problema, enfoque necesario para los problemas más difíciles del diseño.

Describe una relación: los patrones no sólo describen módulos, sino estructuras y mecanismos más profundos del sistema.

El patrón tiene un componente humano significativo (minimiza la intervención humana). Todo el software sirve para el confort humano o la calidad de vida; los mejores patrones recurren explícitamente a la estética y a la utilidad.

Clasificación

Los **patrones creacionales** se centran en la “creación, composición y representación” de objetos. hacen la observación de que los patrones creacionales “encierran conocimiento acerca de cuáles son las clases concretas que usa el sistema”, pero al mismo tiempo “ocultan la forma en la que las instancias de dichas clases se crean y agrupan”. Los patrones creacionales ofrecen mecanismos que hacen más fácil la formación de las instancias de los objetos dentro de un sistema y establecen “restricciones en el tipo y número de objetos que es posible crear dentro de un sistema”

Los **patrones estructurales** se centran en problemas y soluciones asociados con la manera en la que se organizan e integran las clases y objetos para construir una estructura más grande. En esencia, ayudan a establecer relaciones entre entidades dentro de un sistema. Por ejemplo, los patrones estructurales que se centran en aspectos orientados a clases proporcionan mecanismos de herencia que conducen a interfaces de programa más eficaces.

Los **patrones estructurales** que se centran en objetos sugieren técnicas para combinar objetos dentro de otros objetos o para integrarlos en una estructura más amplia. Los patrones conductuales se enfocan a problemas asociados con la asignación de responsabilidad entre los objetos y a la manera en la que se efectúa la comunicación entre ellos.



Patrones creacionales, estructurales y conductuales

En la red mundial hay una amplia variedad de patrones de diseño que se ajustan a las categorías creacional, estructural y conductual. En Wikipedia (www.wikipedia.org) se encuentran los siguientes ejemplos:

Patrones creacionales

- **Patrón de fábrica abstracta:** centraliza la decisión acerca de para qué fábrica deben hacerse instancias.
- **Patrón de método de fabricación:** centraliza la creación de un objeto de tipo específico para elegir una entre varias implementaciones.
- **Patrón constructor:** separa la construcción de un objeto complejo a partir de su representación, de modo que el mismo proceso de construcción pueda crear representaciones distintas.
- **Patrón prototipo:** se usa cuando el costo inherente que implica crear un nuevo objeto en la forma estándar (como con el empleo de una “nueva” clave) es prohibitivo para una aplicación dada.
- **Patrón de instancia única:** restringe la formación de instancias de una clase a un objeto.

Patrones estructurales

- **Patrón adaptador:** “adapta” una interfaz para una clase en otra que espera un cliente.
- **Patrón agregado:** es una versión del patrón compuesto con métodos para agregar hijos.
- **Patrón de puente:** desacopla una abstracción de su implementación, de modo que las dos puedan variar en forma independiente.
- **Patrón compuesto:** estructura de árbol de objetos en los que cada uno tiene la misma interfaz.

INFORMACIÓN

- **Patrón contenedor:** crea objetos con el único propósito de que sostengan a otros y los administren.
- **Patrón de proximidad:** clase que funciona como interfaz respecto de otra cosa.
- **Tubos y filtros:** cadena de procesos en los que la salida de cada uno es la entrada del siguiente.

Patrones conductuales

- **Cadena de patrones de responsabilidad:** objetos de comando que son manejados o pasados a otros objetos por medio de otros que contienen procesamiento lógico.
- **Patrón de comando:** objetos de comando que encierran una acción y sus parámetros.
- **Escucha de eventos:** se distribuyen datos a objetos registrados para recibirlos.
- **Patrón intérprete:** implementa un lenguaje de computadora especializado para resolver con rapidez un conjunto específico de problemas.
- **Patrón iterador:** los iteradores se utilizan para acceder en forma secuencial a los elementos de un agregado sin exponer su representación subyacente.
- **Patrón mediador:** proporciona una interfaz unificada a un conjunto de interfaces en un subsistema.
- **Patrón visitante:** forma de separar un algoritmo de un objeto.
- **Patrón visitante de un solo servicio:** optimiza la implementación de un visitante que se haya asignado, utilizado sólo una vez y luego eliminado.
- **Patrón visitante jerárquico:** brinda una forma de visitar cada nodo en una estructura jerárquica de datos, como un árbol.

Las descripciones detalladas de estos patrones pueden obtenerse por medio de vínculos en www.wikipedia.org.

Ventajas

La clave para la reutilización es anticiparse a los nuevos requisitos y cambios, de modo que los sistemas evolucionen de forma adecuada.

Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente de otros aspectos.

Un patrón es un esquema o microarquitectura que supone una solución a problemas (dominios de aplicación) semejantes.

Los patrones facilitan la reusabilidad, extensibilidad y mantenimiento.

Unidad 2: Implementación y Pruebas.

Tema 6. Implementación y Evolución.

Técnicas de transformación del diseño al código. Creación de las definiciones de las clases a partir de los DCDs. Creación de métodos a partir de los diagramas de interacción. Resumen de transformación del diseño al código. Estándares de programación y procedimientos. Pautas para la programación. Documentación. **Evolución del software.** Dinámica de evolución de los programas. Mantenimiento del software. Predicción del mantenimiento. Procesos de evolución. Reingeniería de sistemas. Evolución de sistemas heredados

El proceso unificado define el Modelo de Implementación. Éste contiene los artefactos de implementación como el código fuente, las definiciones de bases de datos, las páginas JSP/XML/HTML, etcétera. Por tanto, el código que se va a crear en este capítulo forma parte del Modelo de Implementación.

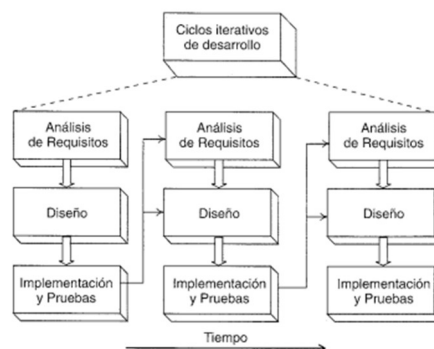
El trabajo de diseño anterior no debería implicar que no exista prototipado o diseño durante la programación; las herramientas de desarrollo modernas proporcionan un entorno excelente para explorar rápidamente enfoques alternativos, y normalmente merece la pena elaborar algo (o incluso mucho) de diseño mientras se programa.

Sin embargo, algunos desarrolladores encuentran útil anticipar un poco mediante el modelado visual antes de programar, especialmente aquellos que se encuentran cómodos con el razonamiento visual o los lenguajes basados en diagramas.

Creatividad y cambio durante la implementación

El trabajo de programación no es una etapa de generación de código trivial, más bien lo contrario. En realidad, los resultados generados durante el diseño son un primer paso incompleto; durante la programación y pruebas, se realizarán innumerables cambios y se descubrirán y resolverán problemas complicados.

Una ventaja del proceso de desarrollo iterativo e incremental es que los resultados de la iteración anterior pueden proporcionar los datos para el comienzo de la siguiente iteración. Por tanto, los resultados del análisis y diseño actual se están refinando y enriqueciendo continuamente a partir del trabajo de implementación anterior.



Creación de las definiciones de las clases a partir de los DCDs

Como mínimo, los DCDs describen los nombres de las clases o interfaces, las superclases, signatura de los métodos y los atributos simples de una clase. Esto es suficiente para crear una definición de clase básica en un lenguaje de programación orientado a objetos.

Definición de una clase con métodos y atributos simples

A partir del DCD, la transformación de las definiciones de los atributos básicas y las signaturas de los métodos de la clase a la definición del lenguaje de programación es directa. A menudo se excluye de los diagramas de clases el método create puesto que es común que aparezca y tiene múltiples interpretaciones, dependiendo del lenguaje que se vaya a utilizar.

Añadir atributos de referencia

Los atributos de referencia de una clase se deducen de las asociaciones y la navegabilidad de un diagrama de clases, es decir, un atributo que referencia a otro objeto complejo,

Atributos de referencia y los nombres de los roles

Cada extremo de asociación se denomina rol. Brevemente, un nombre de rol es un nombre que identifica al rol y, a menudo, proporciona algo del contexto semántico acerca de la naturaleza del rol. Si el nombre de un rol está presente en un diagrama de clases, utilícelo como base para el nombre del atributo de referencia durante la generación de código.



Manejo de excepciones y de errores

El desarrollo de la aplicación es aconsejable tener en cuenta el manejo de las excepciones durante el trabajo de diseño y, por supuesto, durante la implementación. Brevemente, en UML, las excepciones se representan como mensajes asíncronos en los diagramas de interacción.

Programar probando primero

Una práctica excelente promovida por el método de Programación Extrema es programar probando primero. En esta práctica, se escribe el código de las pruebas de unidad antes del código que se va a probar, y los desarrolladores escriben el código de las pruebas de unidad para todo el código de producción.

Resumen de la transformación de los diseños en código

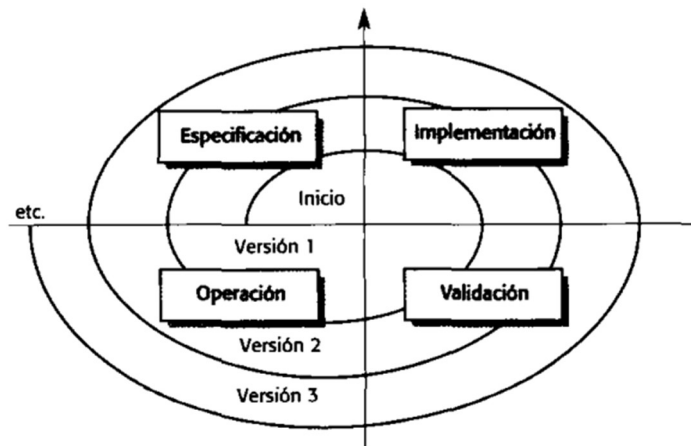
El proceso de traducción de los DCD a las definiciones de las clases, y de los diagramas de interacción en los métodos, es relativamente directo. Durante el trabajo de programación todavía hay cabida para tomar decisiones, realizar cambios en el diseño y explorar, pero algunas de las grandes ideas del diseño se tuvieron en cuenta antes de la programación.

Evolución de Software

Después de que los sistemas hayan sido desarrollados, inevitablemente han de sufrir cambios si tienen que seguir siendo útiles. Una vez que el software comienza a utilizarse, surgen nuevos requerimientos y los requerimientos existentes cambian. Los cambios en los negocios a menudo generan nuevos requerimientos para el software existente.

La evolución del software es importante debido a que las organizaciones actualmente son completamente dependientes de sus sistemas software y han invertido millones de dólares en ellos. Los sistemas software son activos de negocio críticos y las organizaciones deben invertir en los cambios del sistema para mantener el valor de estos activos. Por lo tanto, en grandes compañías, la mayor parte del presupuesto de software se dedica a mantener los sistemas existentes, y no debería sorprender que algunas figuras como las de Erlikh (Erlikh, 2(00) sugieran que el 90% de los costes del software sean costes de evolución

La mayoría de los cambios son una consecuencia de que se generan nuevos requerimientos como respuesta a cambios en el negocio y en las necesidades de los usuarios. Como consecuencia, se puede pensar en la ingeniería del software como un proceso en espiral con requerimientos, diseño, implementación y pruebas que se realizan continuamente durante el tiempo de vida del sistema.



Dinámica de evolución de los programas

Lehman y Belady examinaron el crecimiento y la evolución de varios sistemas software grandes. Propusieron las siguientes leyes

La primera ley establece que el mantenimiento de los sistemas es un proceso inevitable. A medida que el entorno del sistema cambia, surgen nuevos requerimientos y el sistema debe ser modificado. Cuando el sistema modificado vuelve a introducirse en el entorno, éste promueve más cambios en el entorno, de forma que el proceso de evolución se recicla

La segunda ley establece que, a medida que se cambia un sistema, su estructura se degrada. La única forma de evitar este hecho es invertir en mantenimiento preventivo en el que se consume tiempo mejorando la estructura del software sin añadir funcionalidades.

La tercera ley, quizás, la más interesante y discutida de las leyes de Lehman. Sugiere que los grandes sistemas tienen su propia dinámica que se establece en una etapa temprana en el proceso de desarrollo. Ésta determina las tendencias generales del proceso de mantenimiento del sistema y limita el número de posibles cambios en el sistema.

La cuarta ley de Lehman sugiere que la mayoría de los proyectos de programación grandes trabajan en lo que se denomina un estado saturado. Es decir, un cambio en los recursos o en el personal tiene efectos imperceptibles en la evolución a largo plazo del sistema. Esto concuerda con la tercera ley, que sugiere que la evolución del programa es independiente en gran medida de las decisiones de gestión. Esta ley confirma que los grandes grupos de desarrollo software son a menudo improductivos debido a que las sobrecargas en las comunicaciones dominan el trabajo del grupo.

La quinta ley de Lehman se refiere a los incrementos de los cambios en cada entrega del sistema. Añadir nueva funcionalidad a un sistema inevitablemente introduce nuevos defectos en el sistema. Cuanta más funcionalidad se añada en cada entrega, más defectos habrá. La ley sugiere que no deberían presupuestarse incrementos grandes de funcionalidad en cada entrega sin tener en cuenta la necesidad de reparación de defectos.

Mantenimiento del software

El mantenimiento del Software: es el proceso general de cambiar un sistema después de que éste ha sido entregado. El término se aplica normalmente a software a medida en donde grupos de desarrollo distintos están implicados antes y después de la entrega. Los cambios realizados al software pueden ser cambios sencillos para corregir errores de código. cambios más extensos para corregir errores de diseño o mejoras significativas.

Existen tres tipos diferentes de mantenimiento de software:

Mantenimiento para reparar defectos del software. Por lo general. los errores de código son relativamente baratos de corregir; los errores de diseño son mucho más caros ya que implican reescribir varios componentes de los programas.

Mantenimiento para adaptar el software a diferentes entornos operativos. Este tipo de mantenimiento se requiere cuando cambia algún aspecto del entorno del sistema. como por ejemplo el hardware. la plataforma del sistema operativo u otro software de soporte. El sistema de aplicaciones debe modificarse para adaptarse a estos cambios en el entorno.

Mantenimiento para añadir o modificarlas funcionalidades del sistema. Este tipo de mantenimiento es necesario cuando los requerimientos del sistema cambian como respuesta a cambios organizacionales o del negocio. La escala de los cambios requeridos en el software es a menudo mucho mayor que en los otros tipos de mantenimiento

En la práctica, no existe una clara distinción entre estos tipos de mantenimiento. Cuando se adapta el sistema a un nuevo entorno. pueden añadirse funcionalidades para aprovechar las ventajas de las nuevas características del entorno. Los defectos del software a menudo se muestran debido a que los usuarios utilizan el sistema en formas no anticipadas. La mejor forma de reparar estos defectos consiste en realizar cambios en el sistema que se adapten a su forma de trabajo

Normalmente se reconocen estos tipos de mantenimiento, pero varias personas les dan distintos nombres. El mantenimiento correctivo se utiliza generalmente para referirse al mantenimiento para reparación de defectos. Sin embargo, el mantenimiento adaptativo algunas veces significa adaptarse a un nuevo entorno y puede significar adaptar el software a nuevos requerimientos. El mantenimiento perfectivo puede significar perfeccionar el software implementando nuevos requerimientos; en otros casos significa mantener la funcionalidad del sistema, pero mejorando su estructura y su rendimiento.

Coste de Mantenimiento

Una razón importante de por qué los costes de mantenimiento son altos es que es más caro añadir funcionalidades después de que el sistema esté en funcionamiento que implementar la misma funcionalidad durante el desarrollo. Los factores clave que distinguen el desarrollo y el mantenimiento, y que conducen a costes de mantenimiento más elevados, son:

Estabilidad del equipo. Después de entregar un sistema, es normal que el equipo de desarrollo se disuelva y la gente trabaje en nuevos proyectos. El nuevo equipo o los individuos responsables del mantenimiento del sistema no comprenden dicho sistema o las razones de fondo de las decisiones de su diseño.

Responsabilidad contractual. El contrato para mantener un sistema normalmente está separado del contrato para desarrollar el sistema. El contrato de mantenimiento puede darse con una compañía diferente en lugar de con el desarrollador original del sistema. Este factor, junto con la ausencia de estabilidad del equipo, implica que no existe incentivo para que un equipo de desarrollo escriba el software para que sea fácil de cambiar.

Habilidades del personal. El personal de mantenimiento a menudo no tiene experiencia y no está familiarizado con el dominio de la aplicación. El mantenimiento tiene una pobre imagen entre los ingenieros software.

Edad y estructura del programa. A medida que pasa el tiempo, la estructura de los programas tiende a degradarse con los cambios, por lo que se vuelve más difícil de comprender y modificar. Algunos sistemas han sido desarrollados sin técnicas modernas de ingeniería del software.

Normalmente resulta rentable invertir **esfuerzo en el diseño e implementación de un sistema para reducir los costes de mantenimiento**. Añadir nuevas funcionalidades después de la entrega es caro debido a que hay que emplear tiempo en la comprensión del sistema y en el análisis del impacto de los cambios propuestos. Por lo tanto, el trabajo realizado durante el desarrollo para hacer que el software sea más fácil de entender y de cambiar, probablemente reduzca los costes de mantenimiento. Buenas técnicas de ingeniería del software tales como una especificación precisa, el uso de desarrollo orientado a objetos y gestión de configuraciones contribuyen a la reducción de los costes de mantenimiento.

Predicción del mantenimiento

Se debe intentar predecir qué cambios del sistema son probables y qué partes del sistema son probablemente las más difíciles de mantener. También se debería intentar estimar los costes totales de mantenimiento para un sistema durante un periodo de tiempo determinado. Como es obvio, estas predicciones están estrechamente relacionadas:

La aceptación o no de un cambio en el sistema depende, hasta cierto punto, de la mantenibilidad de los componentes del sistema afectados por dicho cambio.

La implementación de los cambios del sistema tiende a degradar la estructura de dicho sistema y, por lo tanto, reduce su mantenibilidad.

Los costes de mantenimiento dependen del número de cambios, y los costes de la implementación de los cambios dependen de la mantenibilidad de los componentes del sistema.

Para evaluar las relaciones entre un sistema y su entorno, debería tenerse en cuenta:

El número y la complejidad de las interfaces del sistema. Cuanto mayor sea el número de interfaces y más complejas sean éstas, es más probable que se hagan más peticiones de cambio.

El número de requerimientos del sistema intrínsecamente volátiles. Los requerimientos que reflejan políticas y procedimientos organizacionales son probablemente más volátiles que los requerimientos que se basan en características estables del dominio.

Los procesos de negocios en los que se utiliza el sistema. Puesto que los procesos de negocios evolucionan, generan peticiones de cambio del sistema. Cuantos más procesos de negocios utilice el sistema, habrá más peticiones de cambio del sistema.

Después de que el sistema se haya puesto en funcionamiento, se pueden usar los datos del proceso para ayudar a predecir la mantenibilidad. Ejemplos de métricas del proceso que pueden utilizarse para evaluar la mantenibilidad son:

El número de peticiones de mantenimiento correctivo. Un crecimiento en el número de informes de fallos de ejecución puede indicar que se han introducido más errores en el programa de los que se han corregido durante el proceso de mantenimiento. Esto puede suponer una disminución de la mantenibilidad.

El tiempo medio requerido para el análisis de impacto. Éste refleja el número de componentes del programa que se ven afectados por una petición de cambio. Si este tiempo se incrementa, implica que más y más componentes se ven afectados y que la mantenibilidad disminuye.

El tiempo medio empleado en implementar una petición de cambio. Éste no es el mismo que el tiempo para el análisis de impacto, aunque está relacionado con él. Se trata de la cantidad de tiempo que se necesita para modificar realmente el sistema y su documentación, después de que haya evaluado qué componentes se ven afectados.

El número de peticiones de cambio pendientes. Un incremento en este número a lo largo del tiempo puede implicar una disminución en la mantenibilidad.

Procesos de evolución

Los procesos de evolución del software varían considerablemente dependiendo del tipo de software a mantener, los procesos de desarrollo utilizados en una organización y el personal implicado en el proceso. En algunas organizaciones, la evolución puede ser un proceso informal en el que la mayor parte de las peticiones de cambios surgen de conversaciones entre los usuarios del sistema y los desarrolladores.

Los procesos de evolución incluyen las actividades fundamentales de análisis de cambios, planificación de entregas, implementación del sistema y entrega de un sistema a los clientes. Se evalúa el coste e impacto de estos cambios para ver en qué medida se ve afectado el sistema por el cambio y cuánto cuesta implementar el cambio. Si los cambios propuestos son aceptados, se planifica una nueva entrega del sistema.

El proceso de implementación de los cambios es, esencialmente, una iteración del proceso de desarrollo en la que se diseñan, implementa y prueban las revisiones del sistema. Sin embargo, una diferencia fundamental es que la etapa inicial de la implementación de los cambios es la

comprensión del programa. Durante esta fase, se tiene que entender cómo está estructurado el programa y cómo realiza su funcionalidad. Cuando se implementa un cambio, se usa este entendimiento para estar seguro de que el cambio implementado no afecta de forma adversa al sistema existente.

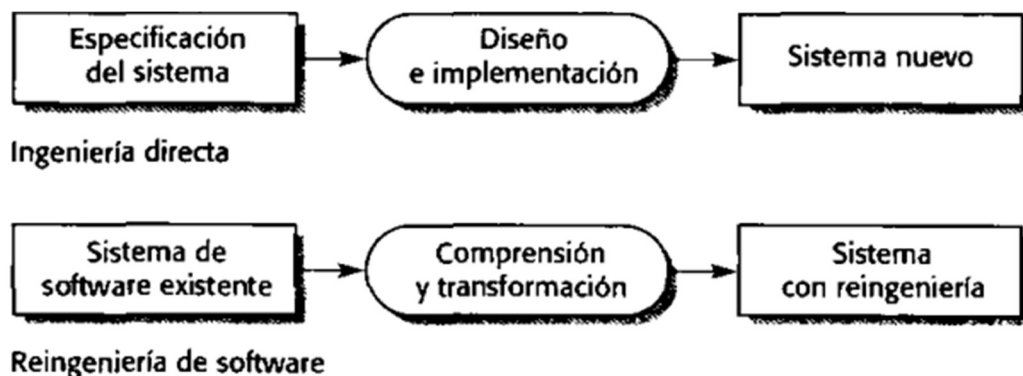
Reingeniería de sistemas

La reingeniería puede implicar redocumentar el sistema, organizar y reestructurar el sistema, traducir el sistema a un lenguaje de programación más moderno, y modificar y actualizar la estructura y valores de los datos del sistema. La funcionalidad del software no se cambia y, normalmente, la arquitectura del sistema también sigue siendo la misma. Hacer reingeniería de un sistema software tiene dos ventajas clave sobre aproximaciones más radicales a la evolución del sistema:

Riesgo reducido. Existe un alto riesgo en volver a desarrollar software crítico para los negocios. Pueden cometerse errores en la especificación, o puede haber problemas en el desarrollo. Los retrasos en la introducción del nuevo software pueden significar pérdidas en el negocio e incurrir en costes adicionales.

Coste reducido. El coste de hacer reingeniería es significativamente menor que el coste de desarrollar nuevo software.

La distinción crítica entre reingeniería y nuevo desarrollo software es el punto de partida del desarrollo. En lugar de empezar con una especificación escrita, el sistema antiguo actúa como una especificación para el nuevo sistema. Chikofsky y Cross denominan al desarrollo convencional ingeniería hacia adelante para distinguirla de la reingeniería del software.



La entrada del proceso es un programa heredado y la salida es una versión modularizada y estructurada del mismo programa. Durante la reingeniería del programa, los datos del sistema también sufren reingeniería. Las actividades de este proceso de reingeniería son:

Traducción del código fuente. El programa es convertido desde un lenguaje de programación antiguo a una versión más moderna del mismo lenguaje o a un lenguaje diferente.

Ingeniería inversa. El programa se analiza y se extrae información a partir de él. Esto ayuda a documentar su organización y funcionalidad.

Mejora de la estructura de los programas. La estructura de control del programa se analiza y modifica para hacerla más fácil de leer y comprender.

Modularización de los programas. Se agrupan las partes relacionadas del programa y se elimina la redundancia en donde resulta adecuado.

Reingeniería de datos. Los datos procesados por el programa se cambian para reflejar los cambios en él.

Además de la amplitud de la reingeniería, los principales factores que afectan a los costes de reingeniería son:

La calidad del software sobre el que se va a hacer reingeniería. Cuanto más baja sea la calidad del software y su documentación asociada (si la hay), más altos serán los costes de reingeniería

Las herramientas de soporte disponibles para la reingeniería. Normalmente no es rentable hacer reingeniería sobre un sistema software a menos que puedan utilizarse herramientas CASE para automatizar la mayor parte de los cambios en los programas

La amplitud de la conversión de datos requerida. Si el sistema sobre el que se va a hacer reingeniería requiere que se conviertan grandes volúmenes de datos, el coste del proceso se incrementa de forma significativa.

La disponibilidad de personal experto. Si el personal responsable de mantener el sistema no puede implicarse en el proceso de reingeniería, los costes se incrementarán debido a que los ingenieros encargados de la reingeniería tienen que invertir una gran cantidad de tiempo en comprender el sistema.

Evolución de sistemas heredados

Las organizaciones que cuentan con un presupuesto limitado para mantener y actualizar sus sistemas heredados tienen que decidir cómo obtener los mejores beneficios a su inversión. Esto significa que tienen que realizar evaluaciones realistas de sus sistemas heredados y a continuación decidir cuál es la estrategia más adecuada para la evolución de estos sistemas. Existen cuatro opciones estratégicas:

Desechar completamente el sistema. Esta opción debería elegirse cuando el sistema no constituye una contribución efectiva para los procesos de negocio. Esto ocurre cuando los procesos de negocio han cambiado desde que se instaló el sistema y ya no son completamente dependientes de éste. Ejemplo cuando un sistema operativo queda obsoleto y hay que migrar a uno más moderno.

Dejar el sistema sin cambios y continuar con un mantenimiento regular. Esta opción debería elegirse cuando el sistema todavía es necesario, pero es muy estable y los usuarios del sistema solicitan un número relativamente pequeño de peticiones de cambio.

Hacer reingeniería del sistema para mejorar su mantenibilidad. Esta opción debería elegirse cuando la calidad del sistema se ha degradado por los cambios continuos y cuando dichos cambios todavía son necesarios. Cuando la versión de un lenguaje quedo deprecado y hay que migrar a nuevas versiones por políticas de seguridad.

Reemplazar todo o parte del sistema con un nuevo sistema. Esta opción debería elegirse cuando otros factores, como un nuevo hardware, implican que el sistema antiguo no puede continuar en funcionamiento o cuando los sistemas comerciales deberían permitir desarrollar el nuevo sistema con un coste razonable.

Desde una perspectiva de negocio, tiene que decidirse si el negocio realmente necesita del sistema. Desde una perspectiva técnica, tiene que evaluarse la calidad de la aplicación software y del software y hardware de soporte del sistema. Para evaluar el valor de negocio de un sistema, se tiene que identificar a los stakeholders del sistema, tales como usuarios finales del sistema y sus gestores, y plantearles una serie de cuestiones sobre dicho sistema. Existen cuatro cuestiones básicas que deberían abordarse:

El uso del sistema. Si los sistemas sólo se utilizan de forma ocasional por un pequeño número de personas, pueden tener un bajo valor de negocio.

Los procesos de negocio que están soportados. Cuando se introduce un sistema, pueden diseñarse procesos de negocio para explotar ese sistema. Sin embargo, cambiar estos procesos puede resultar imposible debido a que el sistema heredado no puede ser adaptado.

Confiabilidad del sistema, La confiabilidad del sistema no sólo es un problema técnico sino también un problema de negocio. Si un sistema no es confiable y los problemas afectan directamente a los clientes del negocio o suponen que las personas del negocio son apartadas de otras tareas para resolver estos problemas, el sistema tiene un bajo valor de negocio.

Salidas del sistema, La clave fundamental aquí es la importancia de las salidas del sistema para un funcionamiento con éxito del negocio. Si el negocio depende de estas salidas, entonces el sistema tiene un alto valor de negocio.

Tema 7. Verificación y Pruebas del Sistema.

Verificación y validación. Planificación de la verificación y validación. Inspecciones de software. Análisis estático automatizado. Pruebas del software: Pruebas de Caja blanca: camino básico, bucles. Pruebas de Caja negra: partición equivalente, análisis de valores límites. Estrategias de Prueba. Defectos y Fallas. Diseño de casos de prueba. Pruebas de Integración. Pruebas de Regresión. La importancia de las pruebas en el cronograma del proyecto. Verificación, validación y calidad de producto

Durante y después del proceso de implementación, el programa que se está desarrollando debe ser comprobado para asegurar que satisface su especificación y entrega la funcionalidad esperada por las personas que pagan por el software. La verificación y la validación (V & V) es el nombre dado a estos procesos de análisis y pruebas. La verificación y la validación tienen lugar en cada etapa del proceso del software. V & V comienza con revisiones de los requerimientos y continúa con revisiones del diseño e inspecciones de código hasta la prueba del producto.

«Validación: ¿Estamos construyendo el producto correcto?»

«Verificación: ¿Estamos construyendo el producto correctamente?»

La verificación se refiere al conjunto de tareas que garantizan que el software implementa correctamente una función específica. La validación es un conjunto diferente de tareas que aseguran que el software que se construye sigue los requerimientos del cliente.

El objetivo último del proceso de verificación y validación es establecer la seguridad de que el sistema software está «hecho para un propósito». Esto significa que el sistema debe ser lo suficientemente bueno para su uso pretendido. El nivel de confianza requerido depende del propósito del sistema, las expectativas de los usuarios del sistema y el entorno de mercado actual del sistema:

Función del software. El nivel de confianza requerido depende de lo crítico que sea el software para una organización. Por ejemplo, el nivel de confianza requerido para el software que se utiliza para controlar un sistema de seguridad crítico es mucho más alto que el requerido para un prototipo de un sistema software que ha sido desarrollado para demostrar algunas ideas nuevas.

Expectativas del usuario. Una reflexión lamentable sobre la industria del software es que muchos usuarios tienen pocas expectativas sobre su software y no se sorprenden cuando éste falla durante su uso. Están dispuestos a aceptar estos fallos del sistema cuando los beneficios de su uso son mayores que sus desventajas. Actualmente es menos aceptable entregar sistemas no fiables, por lo que las compañías de software deben invertir más esfuerzo para verificar y validar.

Entorno de mercado. Cuando un sistema se comercializa, los vendedores del sistema deben tener en cuenta los programas competidores, el precio que sus clientes están dispuestos a pagar por el sistema y la agenda requerida para entregar dicho sistema. Cuando una compañía tiene pocos competidores, puede decidir entregar un programa antes de que haya sido completamente probado y depurado, debido a que quiere ser el primero en el mercado.

Las inspecciones de software analizan y comprueban las representaciones del sistema tales como el documento de requerimientos, los diagramas de diseño y el código fuente del programa. Puede usarse las inspecciones en todas las etapas del proceso. Las inspecciones pueden ser complementadas con algún tipo de análisis automático del código fuente de un sistema o de los documentos asociados.

Las pruebas del software implican ejecutar una implementación del software con datos de prueba. Se examinan las salidas del software y su entorno operacional para comprobar que funciona tal y como se requiere. Las pruebas son una técnica dinámica de verificación y validación.

Sólo puede probarse un sistema cuando está disponible un prototipo o una versión ejecutable del programa. Una ventaja del desarrollo incremental es que una versión probable del sistema está disponible en etapas tempranas del proceso de desarrollo. Las funcionalidades pueden probarse a medida que se van añadiendo al sistema, por lo que no tiene que realizarse una implementación completa antes de que comiencen las pruebas.

Las técnicas de inspección comprenden las inspecciones de programas, el análisis automático del código fuente y la verificación formal. Sin embargo, las técnicas estáticas sólo pueden comprobar la correspondencia entre un programa y su especificación (verificación); no pueden demostrar que el software es operacionalmente útil.

Existen dos tipos distintos de pruebas que pueden utilizarse en diferentes etapas del proceso del software:

Las pruebas de validación intentan demostrar que el software es el que el cliente quiere ~ue satisface sus requerimientos-o Como parte de la prueba de validación, se pueden utilizar pruebas estadísticas para probar el rendimiento y la fiabilidad de los programas. y para comprobar cómo trabaja en ciertas condiciones operacionales.

Las pruebas de defectos intentan revelar defectos en el sistema en lugar de simular su uso operacional. El objetivo de las pruebas de defectos es hallar inconsistencias entre un programa y su especificación.

Normalmente, los procesos de V & V y depuración se intercalan. A medida que se descubren defectos en el programa que se está probando, tiene que cambiarse éste para corregir tales defectos. Sin embargo, las pruebas (o más generalmente la verificación y validación) y la depuración tienen diferentes objetivos:

Los procesos de verificación y validación intentan establecer la existencia de defectos en el sistema software.

La depuración es un proceso (Figura 22.2) que localiza y corrige estos defectos.

Las herramientas de depuración interactivas generalmente forman parte de un conjunto de herramientas de soporte del lenguaje que se integran con un sistema de compilación. Ésta.\ proporcionan un entorno de ejecución especializado para el programa que permite acceder a la tabla de símbolos del compilador y, desde aquí, a los valores de las variables del programa.

Debería poderse establecer una traza entre los casos de prueba y los componentes que son probados. Si esta trazabilidad se documenta, entonces se puede ejecutar un subconjunto de los casos de prueba del sistema para comprobar el componente modificado y sus dependientes.

Planificación de la verificación y validación

La verificación y validación es un proceso caro. Para algunos sistemas, tales como los sistemas de tiempo real con restricciones no funcionales complejas, más de la mitad del presupuesto para el desarrollo del sistema puede invertirse en V & V. Es necesaria una planificación cuidadosa para obtener el máximo provecho de las inspecciones y pruebas y controlar los costes del proceso de verificación y validación.

Debería comenzarse la planificación de la verificación y validación del sistema en etapas tempranas del proceso de desarrollo.

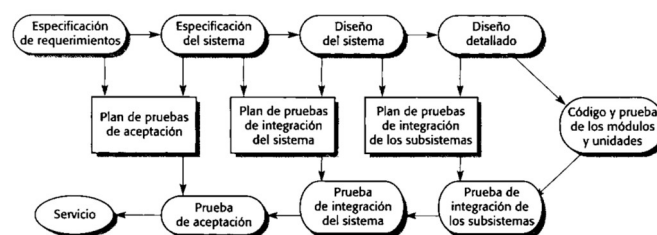


Figura 22.3 Planes de pruebas como un enlace entre las pruebas y el desarrollo.

Como parte del proceso de planificación de v & v, habría que decidir un equilibrio entre las aproximaciones estáticas y dinámicas de la verificación y validación, y pensar en estándares y procedimientos para las inspecciones y pruebas del software, establecer listas de comprobación para conducir las inspecciones de programas y definir el plan de pruebas del software.

La planificación de las pruebas está relacionada con el establecimiento de estándares para el proceso de las pruebas, no sólo con la descripción de los productos de las pruebas. Los planes de pruebas, además de ayudar a los gestores a asignar recursos y estimar el calendario de las pruebas. son de utilidad para los ingenieros del software implicados en el diseño y la realización de las pruebas del sistema. Éstos ayudan al personal técnico a obtener una panorámica general de las pruebas del sistema y ubicar su propio trabajo en este contexto.

Para sistemas más pequeños, se puede utilizar un plan de pruebas menos formal, pero sigue siendo necesario un documento formal para soportar la planificación del proceso de pruebas. Para algunos procesos ágiles como la programación extrema. las pruebas son inseparables del desarrollo. Al igual que otras actividades de planificación, la planificación de las pruebas también es incremental

Los planes de pruebas no son documentos estáticos. sino que evolucionan durante el proceso de desarrollo, Los planes de pruebas cambian debido a retrasos en otras etapas del proceso de desarrollo.

Los principales componentes de un plan de pruebas para un sistema grande y complejo se muestran en la siguiente figura

El proceso de prueba

Una descripción de las principales fases del proceso de prueba. Éstas podrían describirse como se hizo anteriormente en este capítulo.

Trazabilidad de requerimientos

Los usuarios son los más interesados en que el sistema satisfaga sus requerimientos y las pruebas deberían planificarse para que todos los requerimientos se prueben individualmente.

Elementos probados

Deberían especificarse los elementos del proceso del software que tienen que ser probados.

Calendario de pruebas

Un calendario de todas las pruebas y la asignación de recursos para este calendario se enlaza, obviamente, con la agenda general del desarrollo del proyecto.

Procedimientos de registro de las pruebas

No es suficiente ejecutar simplemente las pruebas; los resultados de las pruebas deben ser registrados sistemáticamente. Debe ser posible auditar el proceso de pruebas para comprobar que se ha llevado a cabo correctamente.

Requerimientos hardware y software

Esta sección debería determinar las herramientas software requeridas y la utilización estimada del hardware.

Restricciones

En esta sección deberían anticiparse las restricciones que afectan al proceso de pruebas como la escasez de personal.

Inspecciones de software

Las inspecciones de software son un proceso de V & V estático en el que un sistema software se revisa para encontrar errores, omisiones y anomalías. Generalmente, las inspecciones se centran en el código fuente, pero puede inspeccionarse cualquier representación legible del software como los requerimientos o un modelo de diseño.

Existen tres ventajas fundamentales de la inspección sobre las pruebas:

Durante las pruebas, los errores pueden enmascarar (ocultar) otros errores. Cuando se descubre un error, nunca se puede estar seguro de si otras anomalías de salida son debidas a un nuevo error o son efectos laterales del error original.

Pueden inspeccionarse versiones incompletas de un sistema sin costes adicionales. Si un programa está incompleto, entonces se necesita desarrollar software de soporte especializado para las pruebas a fin de probar aquellas partes que están disponibles. Esto, obviamente, añade costes al desarrollo del sistema.

Además de buscar los defectos en el programa, una inspección también puede considerar atributos de calidad más amplios de un programa tales como grado de cumplimiento con los estándares, portabilidad y mantenibilidad. Puede buscarse ineficiencias, algoritmos no adecuados y estilos de programación que podrían hacer que el sistema fuese difícil de mantener y actualizar.

Las revisiones y las pruebas tienen cada una sus ventajas e inconvenientes y deberían utilizarse conjuntamente en el proceso de verificación y validación. En realidad, Gilb y Graham sugieren que uno de los usos más efectivos de las revisiones es la revisión de los casos de prueba para un sistema. Las revisiones pueden descubrir problemas con estos tests y ayudar a diseñar formas más efectivas para probar el sistema. Se puede empezar la V & V del sistema con inspecciones en etapas tempranas del proceso de desarrollo, pero una vez que se integra un sistema, se necesita comprobar sus propiedades emergentes y que la funcionalidad del sistema es la que su propietario realmente quiere.

A pesar del éxito de las inspecciones, se ha demostrado que es difícil introducir las inspecciones formales en muchas organizaciones de desarrollo de software. Los ingenieros de software con experiencia en la prueba de programas a menudo son reacios a aceptar que las inspecciones pueden ser más efectivas para detectar defectos que las pruebas.

El proceso de inspección de programas

La diferencia principal entre las inspecciones de programas y otros tipos de revisiones de calidad es que el objetivo primordial de las inspecciones es encontrar defectos en el programa en lugar de considerar cuestiones de diseño más generales. Los defectos pueden ser errores lógicos, anomalías en el código que podrían indicar una condición errónea, o el incumplimiento de los estándares del proyecto o de la organización.

Antes de que comience una inspección del programa, es esencial que:

Se tenga una especificación precisa del código a inspeccionar. Es imposible inspeccionar un componente a un nivel de detalle requerido para detectar defectos sin una especificación completa.

Los miembros del equipo de inspección estén familiarizados con los estándares de la organización.

Se haya distribuido una versión compilable y actualizada del código a todos los miembros del equipo. No existe ninguna razón para inspeccionar código que esté «casi completo») incluso si un retraso provoca desfases en la agenda.

La inspección de programas es un proceso formal realizado por un equipo de al menos cuatro personas. Los miembros del equipo analizan sistemáticamente el código y señalan posibles defectos. En las propuestas originales de Fagan, se sugieren roles tales como autor, lector, probador y moderador.

A continuación de la inspección, el autor del programa debería realizar los cambios para corregir los problemas identificados. En la etapa siguiente, el moderador debería decidir si se requiere una reinspección de código.

Puede decidir que no se requiere una reinspección completa y que los defectos han sido reparados con éxito. El programa entonces es aprobado por el moderador para su entrega.

Durante una inspección, a menudo se utiliza una lista de comprobación de errores de programación comunes para centrar el análisis. Esta lista de comprobación puede basarse en ejemplos de listas de comprobación de libros o en conocimiento de los defectos que son comunes en un dominio de aplicación particular. Se necesitan diferentes listas de comprobación para distintos lenguajes de programación debido a que cada lenguaje tiene sus propios errores característicos

Análisis estático automatizado

Los analizadores estáticos son herramientas software que escanean el código fuente de un programa y detectan posibles defectos y anomalías. Analizan el código del programa y así reconocen los tipos de sentencias en el programa. Pueden detectar si las sentencias están bien formadas, hacer inferencias sobre el flujo de control del programa y, en muchos casos, calcular el conjunto de todos los posibles valores para los datos del programa. Complementan las facilidades de detección de errores proporcionadas por el compilador del lenguaje. Pueden utilizarse como parte del proceso de inspección o como una actividad separada del proceso V & V.

El objetivo del análisis estático automatizado es llamar la atención del inspector sobre las anomalías del programa, tales como variables que se utilizan sin inicialización, variables que no se usan o datos cuyo valor podría estar fuera de alcance. Las anomalías son a menudo el resultado de errores de programación tales como omisiones, de forma que resalten aspectos del programa que podrían funcionar mal. Sin embargo, debería comprenderse que estas anomalías no son necesariamente defectos en el programa. Pueden ser deliberadas o pueden no tener consecuencias adversas.

Las etapas implicadas en el análisis estático comprenden:

Análisis del flujo de control. Esta etapa identifica y resalta bucles con múltiples salidas o puntos de entrada y código no alcanzable.

Análisis del uso de los datos. Esta etapa revela cómo se utilizan las variables del programa. Detecta variables que se utilizan sin inicialización previa, variables que se asignan dos veces y no se utilizan entre asignaciones, y variables que se declaran, pero nunca se utilizan.

Análisis de interfaces. Este análisis comprueba la consistencia de las declaraciones de funciones y procedimientos y su utilización.

Análisis del flujo de información. Esta fase del análisis identifica las dependencias entre las variables de entrada y salida. Mientras no detecte anomalías, muestra cómo se deriva el valor de cada variable del programa a partir de otros valores de variables.

Análisis de caminos. Esta fase del análisis semántico identifica todos los posibles caminos en el programa y muestra las sentencias ejecutadas en dicho camino. Esencialmente desenreda el control del programa y permite que cada posible predicado sea analizado individualmente

Verificación y métodos formales

Los métodos formales de desarrollo del software se basan en representaciones matemáticas del software, normalmente como una especificación formal. Estos métodos formales se ocupan principalmente del análisis matemático de la especificación; de transformar la especificación a una representación más detallada semánticamente equivalente; o de verificar formalmente que una representación del sistema es semánticamente equivalente a otra representación.

Se podría pensar en el uso de métodos formales como la técnica de verificación estática. Los métodos formales requieren un análisis muy detallado de la especificación del sistema y del programa, y su uso consume a menudo tiempo y resulta caro. Como consecuencia, el uso de métodos formales está restringido principalmente a los procesos de desarrollo de software de seguridad crítico y seguro.

Pruebas del software

El objetivo de la etapa de la prueba de componentes es descubrir defectos probando componentes de programas individuales. Estos componentes pueden ser funciones, objetos o componentes reutilizables. Durante las pruebas del sistema, estos componentes se integran para formar subsistemas o el sistema completo. En esta etapa, la prueba del sistema debería centrarse en establecer que el sistema satisface sus requerimientos funcionales y no funcionales.

El proceso de pruebas del software tiene dos objetivos distintos:

Para demostrar al desarrollador y al cliente que el software satisface sus requerimientos.

Para descubrir defectos en el software en que el comportamiento de éste es incorrecto, no deseable o no cumple su especificación.

El primer objetivo conduce a las pruebas de validación, en las que se espera que el sistema funcione correctamente usando un conjunto determinado de casos de prueba que reflejan el uso esperado de aquél. El segundo objetivo conduce a la prueba de defectos, en los que los casos de prueba se diseñan para exponer los defectos.

Las pruebas no pueden demostrar que el software está libre de defectos o que se comportará en todo momento como está especificado. Siempre es posible que una prueba que se haya pasado por alto pueda descubrir problemas adicionales con el sistema.

Los casos de prueba son especificaciones de las entradas para la prueba y la salida esperada del sistema mas una afirmación de lo que se está probando. Los datos de prueba son las entradas que han sido ideadas para probar el sistema. Los datos de prueba a veces pueden generarse automáticamente. La generación automática de casos de prueba es imposible. Las salidas de las pruebas sólo pueden predecirse por personas que comprenden lo que debería hacer el sistema.

Las pruebas tienen que basarse en un subconjunto de posibles casos de prueba. Idealmente, algunas compañías deberían tener políticas para elegir este subconjunto en lugar de dejar esto al equipo de desarrollo. Estas políticas podrían basarse en políticas generales de pruebas, tal como una política en la que todas las sentencias de los programas deberían ejecutarse al menos una vez.

Las pruebas de componentes realizadas por los desarrolladores se basan normalmente en una comprensión intuitiva de cómo los componentes deberían operar. Las pruebas del sistema, sin embargo, tienen que basarse en una especificación escrita del sistema. Ésta puede ser una especificación detallada de requerimientos del sistema o puede ser una especificación orientada al usuario de más alto nivel de las características que debería implementar el sistema.

La mayoría de los tratamientos de pruebas comienzan con las pruebas de componentes y a continuación se realizan las pruebas del sistema.

Pruebas de caja blanca y caja negra

Las pruebas de caja blanca se enfocan en la estructura de control del programa. Los casos de prueba se derivan para asegurar que todos los enunciados en el programa se ejecutaron al menos una vez durante las pruebas y que todas las condiciones lógicas se revisaron. La prueba de ruta o trayectoria básica, una técnica de caja blanca usa gráficos de programa (o matrices gráficas) para derivar el conjunto de pruebas linealmente independientes que garantizarán la cobertura del enunciado. Las pruebas de condición y de flujo de datos revisan aún más la lógica del programa, y la prueba de bucles complementa otras técnicas de caja blanca al proporcionar un procedimiento para revisar los bucles de varios grados de complejidad.

Las pruebas de caja negra se diseñan para validar los requerimientos funcionales sin considerar el funcionamiento interno de un programa. Las técnicas de prueba de caja negra se enfocan en el dominio de

información del software, y derivan casos de prueba mediante la partición de los dominios de entrada y salida de un programa

Pruebas del sistema

Las pruebas del sistema implican integrar dos o más componentes que implementan funciones del sistema o características y a continuación se prueba este sistema integrado. En un proceso de desarrollo iterativo, las pruebas del sistema se ocupan de probar un incremento que va a ser entregado al cliente; en un proceso en cascada, las pruebas del sistema se ocupan de probar el sistema completo.

Para la mayoría de los sistemas complejos, existen dos fases distintas de pruebas del sistema:

Pruebas de integración, en las que el equipo de pruebas tiene acceso al código fuente del sistema. Cuando se descubre un problema, el equipo de integración intenta encontrar la fuente del problema e identificar los componentes que tienen que ser depurados.

Pruebas de entregas, en las que se prueba una versión del sistema que podría ser entregada a los usuarios. Aquí, el equipo de pruebas se ocupa de validar que el sistema satisface sus requerimientos y con asegurar que el sistema es confiable.

Pruebas de integración

La integración del sistema implica identificar grupos de componentes que proporcionan alguna funcionalidad del sistema e integrar éstos añadiendo código para hacer que funcionen conjuntamente.

La principal dificultad que surge durante las pruebas de integración es la localización de los errores. Existen interacciones complejas entre los componentes del sistema, y cuando se descubre una salida anómala, puede resultar difícil identificar dónde ha ocurrido el error. Para hacer más fácil la localización de errores, siempre debería utilizarse una aproximación incremental para la integración y pruebas del sistema.

Prueba de regresión

Cada vez que se agrega un nuevo módulo como parte de las pruebas de integración, el software cambia. Se establecen nuevas rutas de flujo de datos, ocurren nuevas operaciones de entrada/salida y se invoca nueva lógica de control. Dichos cambios pueden causar problemas con las funciones que anteriormente trabajaban sin fallas. En el contexto de una estrategia de prueba de integración, la prueba de regresión es la nueva ejecución de algún subconjunto de pruebas que ya se realizaron a fin de asegurar que los cambios no propagaron efectos colaterales no deseados.

Pruebas de entregas

Las pruebas de entregas son el proceso de probar una entrega del sistema que será distribuida a los clientes. El principal objetivo de este proceso es incrementar la confianza del suministrador en que el sistema satisface sus requerimientos. Si es así, éste puede entregarse como un producto o ser entregado al cliente. Para demostrar que el sistema satisface sus requerimientos, tiene que mostrarse que éste entrega la funcionalidad especificada, rendimiento y confiabilidad, y que no falla durante su uso normal.

Pruebas de rendimiento

Las pruebas de rendimiento tienen que diseñarse para asegurar que el sistema pueda procesar su carga esperada. Esto normalmente implica planificar una serie de pruebas en las que la carga se va incrementando regularmente hasta que el rendimiento del sistema se hace inaceptable.

Las pruebas de estrés van realizando pruebas acercándose a la máxima carga del diseño del sistema hasta que el sistema falla. Este tipo de pruebas tienen dos funciones:

Prueba el comportamiento de fallo de ejecución del sistema. Pueden aparecer circunstancias a través de una combinación no esperada de eventos en donde la carga sobre el sistema supere la máxima carga anticipada.

Sobrecargar el sistema y pueden provocar que se manifiesten defectos que normalmente no serían descubiertos.

Pruebas de interfaces

Las pruebas de interfaces son particularmente importantes para el desarrollo orientado a objetos y basado en componentes. Los objetos y componentes se definen por sus interfaces y pueden ser reutilizados en combinación con otros componentes en sistema") diferentes. Los errores de interfaz en el componente compuesto no pueden detectarse probando los objetos individuales o componentes. Los errores en el componente compuesto pueden surgir debido a interacciones entre sus partes.

Diseño de casos de prueba

El diseño de casos de prueba es una parte de las pruebas de componentes y sistemas en las que se diseñan los casos de prueba (entradas y salidas esperadas) para probar el sistema. El objetivo del proceso de diseño de casos de prueba es crear un conjunto de casos de prueba que sean efectivos descubriendo defectos en los programas y muestren que el sistema satisface sus requerimientos.

Para diseñar un caso de prueba, se selecciona una característica del sistema o componente que se está probando. A continuación, se selecciona un conjunto de entradas que ejecutan dicha característica, documenta las salidas esperadas o rangos de salida y, donde sea posible, se diseña una prueba automatizada que pruebe que las salidas reales y esperadas son las mismas.

Existen varias aproximaciones que pueden seguirse para diseñar casos de prueba:

Pruebas basadas en requerimientos. en donde los casos de prueba se diseñan para probar los requerimientos del sistema. Esta aproximación se utiliza principalmente en la etapa de pruebas del sistema. ya que los requerimientos del sistema normalmente se implementan por varios componentes. Para cada requerimiento. se identifica casos de prueba que puedan demostrar que el sistema satisface ese requerimiento.

Pruebas de particiones. en donde se identifican particiones de entrada y salida y se diseñan pruebas para que el sistema ejecute entradas de todas las particiones y genere salidas en todas las particiones. Las particiones son grupos de datos que tienen características comunes. como todos los números negativos, todos los nombres con menos de 30 caracteres, todos los eventos provocados por la elección de opciones en un menú, y así sucesivamente.

Pruebas estructurales, en donde se utiliza el conocimiento de la estructura del programa para diseñar pruebas que ejecuten todas las partes del programa. Esencialmente, cuando se prueba un programa, debería intentarse ejecutar cada sentencia al menos una vez. Las pruebas estructurales ayudan a identificar casos de prueba que pueden hacer esto posible.

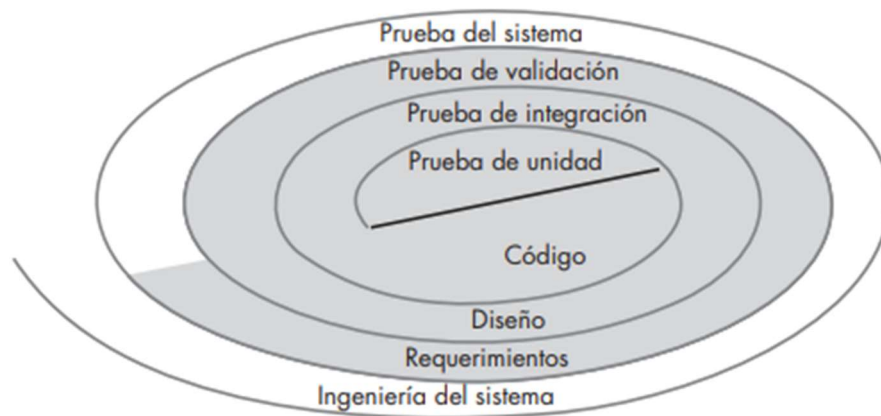
Prueba de Caminos: son una estrategia de pruebas estructurales cuyo objetivo es probar cada camino de ejecución independiente en un componente o programa. Si cada camino independiente. entonces todas las sentencias en el componente deben haberse ejecutado al menos una vez. Además, todas las sentencias condicionales comprueban para los casos verdadero y falso. En un proceso de desarrollo orientado a objetos. pueden utilizarse las pruebas de caminos cuando se prueban los métodos asociados a los objetos.

Automatización de las pruebas

Un banco de pruebas del software es un conjunto integrado de herramientas para soportar el proceso de pruebas. Además de a los marcos de trabajo de pruebas que soportan la ejecución automática de las pruebas. un banco de trabajo puede incluir herramientas para simular otras partes del sistema y generar datos de prueba de dicho sistema.

Normalmente, se necesita una cantidad significativa de esfuerzo y tiempo para crear un banco de trabajo de pruebas adecuado. Por lo tanto, los bancos de trabajo de pruebas completos, sólo se utilizan cuando se

desarrollan sistemas grandes. Para estos sistemas, los costes totales de las pruebas pueden llegar al 50% del total de los costes de desarrollo, por lo que es rentable invertir en herramientas CASE de alta calidad para soportar las pruebas.



Tema 8. Reingeniería.

Conceptos fundamentales. La reingeniería de los procesos del negocio. Principios de reingeniería de procesos de Negocio. Reingeniería del Software. Un modelo de proceso de Reingeniería del software. Ingeniería Inversa. Ingeniería Inversa desde la Interfaz de usuario. Herramientas Case e Ingeniería Inversa.

En la actualidad, existen compañías importantes que poseen decenas de miles de programas de computadora que prestan apoyo a las «viejas reglas del negocio». Cuando los administradores trabajan para modificar estas reglas y lograr así una mayor efectividad y competitividad, el software seguirá el mismo ritmo. En algunos casos, esto implica la creación de sistemas nuevos e importantes basados en computadora'. Pero en muchos otros, esto implica la modificación y/o reconstrucción de las aplicaciones existentes

Cualquier producto de tecnología que está envejeciendo, se rompe con frecuencia, tarda en repararse y ya no representa la última tecnología. ¿Qué se puede hacer? En algunos casos, probablemente lo tirará y se comprará uno nuevo; pero en otros, no dispondrá de la opción de tirarlo. Necesitará reconstruirlo. Creará un producto con una funcionalidad nueva, un mejor rendimiento y fiabilidad, y un mantenimiento mejorado. Eso es lo que llamamos reingeniería.

A nivel de negocio, la reingeniería es ejercida por especialistas de negocio (frecuentemente empresas de consultoría)

A nivel de software, la reingeniería es ejecutada por ingenieros del software.

El mundo sufre constantes cambios. Las demandas de funciones de negocios y de tecnología de información que las soportan cambian a un ritmo que impone mucha presión competitiva en todas las organizaciones comerciales.

¿Cuáles son los pasos?

La reingeniería de procesos de negocios define las metas comerciales, identifica y evalúa los procesos de negocios existentes, y crea procesos comerciales revisados que mejoran las metas actuales. El proceso de reingeniería del software acompaña el análisis de inventarios, la estructuración de documentos, la ingeniería inversa, la reestructuración de datos y la ingeniería avanzada. El objetivo de estas actividades es crear versiones nuevas de los programas existentes que exhiben una mantenibilidad de mayor calidad.

¿Cuál es el producto obtenido?

Son varios los productos que se elaboran (por ejemplo, modelos análisis, modelos de diseño, procedimientos de pruebas). El resultado final es un proceso de reingeniería de negocios y/o el software de reingeniería que lo soporta.

¿Cómo puedo estar seguro de que se he hecho correctamente?

Utilizando las mismas prácticas que se aplican en todos los procesos de ingeniería del software, las revisiones técnicas formales evalúan los modelos de análisis y de diseños, las revisiones especializadas tienen en consideración la capacidad de aplicación comercial y la compatibilidad, y la comprobación se aplica para descubrir los errores en el contenido, en la funcionalidad y en la interoperabilidad.

Principios de reingeniería de procesos de Negocio

La Reingeniería de procesos de negocio, RPN (Business Process Reengineering, BPR2) va más allá del ámbito de las tecnologías de la información y de la ingeniería del software. Entre las muchas definiciones que se han sugerido para la RPN, se cuenta con una publicada en la revista Fortune [STE93]: «. . la búsqueda e implementación de cambios radicales en el proceso de negocios para lograr un avance significativo».

Un proceso de negocio es «un conjunto de tareas lógicamente relacionadas que se llevan a cabo para obtener un determinado resultado de negocio»

Este escenario resulta sumamente conocido: Una aplicación ha dado servicio y ha cubierto las necesidades del negocio de una compañía durante diez o quince años. A lo largo de este tiempo, ha sido corregida, adaptada y mejorada muchas veces. Las personas se dedicaban a esta tarea con la mejor de sus intenciones, pero las prácticas de ingeniería del software buenas siempre se echaban a un lado (por la urgencia de otros problemas). Ahora la aplicación se ha vuelto inestable. Sigue funcionando, pero cada vez que intenta efectuar un cambio se producen efectos colaterales graves e inesperados

En muchos aspectos, la RPN tiene un objetivo y un ámbito idéntico al proceso de la ingeniería de la información. Lo ideal sería que la RPN se produjera de forma descendente, comenzando por la identificación de los objetivos principales del negocio, y culminando con una especificación mucho más detallada de las tareas que definen un proceso específico de negocios.

Un modelo RPN

Definición del negocio. Los objetivos de negocios se identifican en un contexto de cuatro controladores principales: *reducción de costes, reducción de tiempos, mejora de calidad y desarrollo y potenciación del personal*. Los objetivos se pueden definir en el nivel de negocios o para un componente específico del negocio.

Identificación de procesos. En esta actividad se identifican los procesos críticos para alcanzar los objetivos definidos en la definición del negocio. A continuación, pueden recibir prioridades en función de su importancia, necesidad de cambio, o cualquier otra forma que resulte adecuada para la actividad de reingeniería.

Evaluación de procesos. Los procesos existentes deberán analizarse y medirse exhaustivamente. Las tareas de procesos se identificarán; los costes y los tiempos consumidos por las tareas de proceso se anotarán cuidadosamente, y se aislarán los problemas de calidad y rendimiento.

Especificación y diseño de procesos. Basándose en la información obtenida durante las tres primeras actividades de la RPN, se prepararán casos prácticos para cada uno de los procesos que se tengan que rediseñar. Dentro del contexto de la RPN, los casos prácticos identifican un escenario que proporciona resultados a un cliente. Con el uso de casos prácticos como especificación del proceso, se diseña un nuevo conjunto de tareas para el proceso.

Creación de prototipos. Es preciso construir un prototipo del proceso de negocios rediseñado antes de integrarlo por completo en el negocio. Esta actividad «comprueba» el proceso para que sea posible efectuar refinamientos.

Refinamiento e instanciación. Basándose en la realimentación procedente del prototipo, se refina el proceso de negocio y después se instancia en el seno de un sistema de negocio.

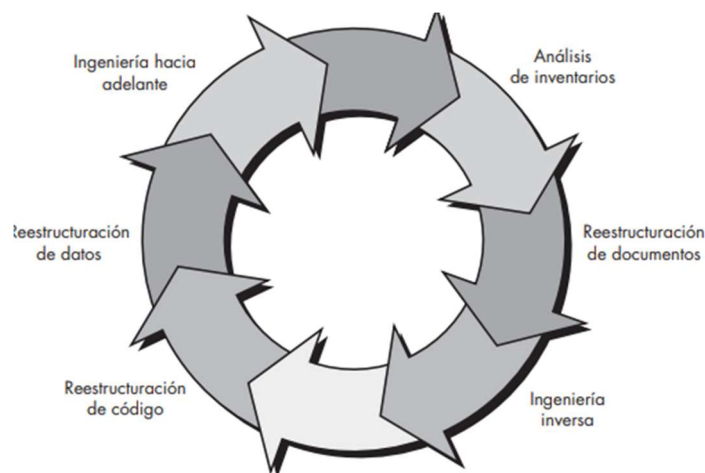
Reingeniería del software

Este escenario resulta sumamente conocido: Una aplicación ha dado servicio y ha cubierto las necesidades del negocio de una compañía durante diez o quince años. A lo largo de este tiempo, ha sido corregida, adaptada y mejorada muchas veces. Las personas se dedicaban a esta tarea con la mejor de sus intenciones, pero las prácticas de ingeniería del software buenas siempre se echaban a un lado (por la urgencia de otros problemas). Ahora la aplicación se ha vuelto inestable. Sigue funcionando, pero cada vez que intenta efectuar un cambio se producen efectos colaterales graves e inesperados. ¿Qué se puede hacer? La imposibilidad de

mantener el software no es un problema nuevo. De hecho, el gran interés por la reingeniería del software ha sido generado por un «iceberg» de mantenimiento de software que lleva creciendo desde hace más de treinta años.

Un modelo de proceso de reingeniería de software

La reingeniería toma tiempo, cuesta cantidades significativas de dinero y absorbe recursos que de otro modo pueden ocuparse en preocupaciones inmediatas. Por todas estas razones, la reingeniería no se logra en pocos meses o incluso en algunos años. La reingeniería de los sistemas de información es una actividad que absorberá recursos de tecnología de la información durante muchos años. Por esto, toda organización necesita una estrategia pragmática para la reingeniería de software.



Ingeniería inversa

La ingeniería inversa puede extraer información de diseño a partir del código fuente, pero el nivel de abstracción, la completitud de la documentación, el grado en el que las herramientas y un analista humano trabajan en conjunto, y la direccionalidad del proceso son enormemente variables

El nivel de abstracción de un proceso de ingeniería inversa y las herramientas usadas para efectuarla tienen que ver con la sofisticación de la información de diseño que puede extraerse del código fuente. De manera ideal, el nivel de abstracción debe ser tan alto como sea posible, es decir, el proceso de ingeniería inversa debe ser capaz de inferir representaciones de diseño procedimental (una abstracción de bajo nivel), información de estructura de programa y datos (un nivel de abstracción un poco más alto), modelos de objeto, modelos de datos y/o flujo de control (un nivel de abstracción relativamente alto) y modelos de relación de entidad (un nivel de abstracción alto)

Ingeniería inversa de interfaces de usuario

Las GUI (interfaces de usuario gráficas) sofisticadas se han vuelto obligatorias para productos y sistemas basados en computadora de todo tipo. Por tanto, el redesarrollo de las interfaces de usuario se ha convertido en uno de los tipos más comunes de actividad de reingeniería. Pero antes de poder reconstruir una interfaz de usuario, debe realizarse ingeniería inversa.

La notación de modelado de comportamiento (capítulo 7) puede proporcionar un medio para desarrollar respuestas a las primeras dos preguntas. Mucha de la información necesaria para crear un modelo de comportamiento puede obtenerse al observar la manifestación externa de la interfaz existente. Pero información adicional necesaria para crear el modelo de comportamiento debe extraerse del código.



Ingeniería inversa

Objetivo: Auxiliar a los ingenieros de software a comprender la estructura interna de diseño de programas complejos.

Mecánica: En la mayoría de los casos, las herramientas de ingeniería inversa aceptan código fuente como entrada y producen varias representaciones de diseño estructural, procedimental, de datos y de comportamiento.

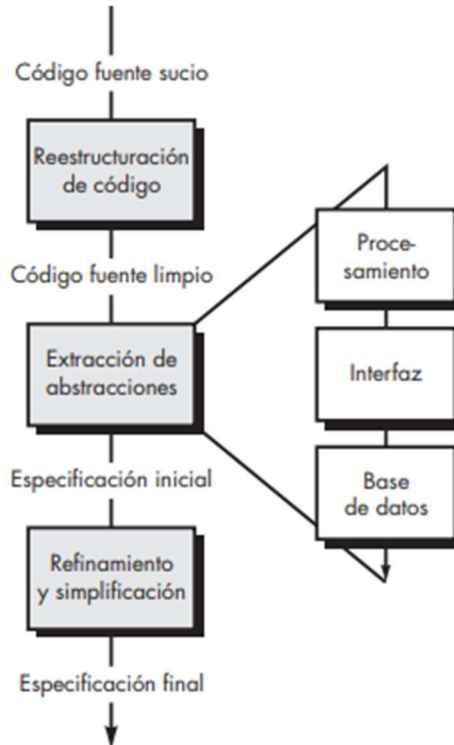
Herramientas representativas:

Imagix 4D, desarrollada por Imagix (www.imagix.com), "ayuda a los desarrolladores de software a comprender software C y C++

complejo o heredado" mediante ingeniería inversa y documentación de código fuente.

Understand, desarrollado por Scientific Toolworks, Inc. (www.scitools.com), analiza gramaticalmente Ada, Fortran, C, C++ y Java "para ingeniería inversa, documenta automáticamente, calcula métricas de código y le ayuda a comprender, navegar y mantener código fuente".

Una lista exhaustiva de herramientas de ingeniería puede encontrarse en <http://scgwiki.iam.unibe.ch:8080/SCG/370>



Tema 9. Gestión de Cambios:

Cambios - Configuración - Liberación. Definir el Plan de Cambios. Problemática de la gestión de Cambios. La importancia de la Gestión de Configuración. Herramientas de soporte a la Gestión de Configuración. Auditorías de la Configuración. Gestión de la Liberación. Conceptos de la Liberación. Seguimiento de la liberación y Control de Conformidad del Usuario. Identificación de los Riesgos. Generación del plan. Estrategias del Riesgo Proactivas o Reactivas. Enfoques. Evaluación de Impacto. Supervisión y Gestión de Riesgos. Normas relacionadas

Los procedimientos de gestión de configuraciones definen cómo registrar y procesar los cambios propuestos al sistema, cómo relacionar éstos con los componentes del sistema y los métodos utilizados para identificar las diversas versiones del sistema. Es necesario gestionar estos sistemas que evolucionan, porque es fácil perder la pista de los cambios que se han incorporado dentro de cada versión. Las versiones incorporan propuestas de cambios, correcciones de fallos y, adaptaciones para hardware y sistemas operativos diferentes.

Hay muchas razones por las que los sistemas existen en diversas configuraciones. Éstas se producen para diferentes computadoras. para diferentes sistemas operativos. para incorporar funciones específicas del cliente. etcétera (véase la Figura 29.1). Los gestores de la configuración son responsables de llevar los registros de las diferencias entre las versiones del software. para asegurar que las nuevas versiones se deriven de forma controlada y para entregar las nuevas versiones a los clientes correctos en el momento justo.

En un proceso tradicional de desarrollo de software basado en el modelo en «cascada» (véase el Capítulo 4), el software se entrega al equipo de gestión de configuraciones después de que el desarrollo haya sido completado y se hayan probado los componentes de software. Luego este equipo tiene la responsabilidad de construir el sistema completo y gestionar las pruebas del sistema. Los fallos encontrados durante las pruebas del sistema se devuelven al equipo de desarrollo para su reparación. A continuación, reparan el fallo y entregan una nueva versión del componente reparado al equipo de la garantía de calidad. Si la calidad es aceptable, éste pasa a ser la nueva línea base para el desarrollo del sistema.

Para ajustarse a este desarrollo incremental, algunas organizaciones han desarrollado un nuevo enfoque para gestionar las configuraciones que permite el desarrollo concurrente y las pruebas del sistema. La principal ventaja de utilizar construcciones diarias del software es que se incrementan las oportunidades de encontrar problemas que surgen a partir de las interacciones al inicio del proceso. Más aún, las construcciones diarias provocan las pruebas de las unidades de los componentes.

La utilización exitosa de las construcciones diarias requiere procesos de gestión del cambio muy rigurosos para llevar un registro de los problemas encontrados y resueltos. También conduce a la gestión de un número muy grande de versiones del sistema y de los componentes.

La gestión de configuraciones en el desarrollo ágil y desarrollo rápido no pueden basarse en rígidos procedimientos y papeleo burocrático. Aunque éstos pueden ser necesarios para proyectos grandes o complejos, pueden ralentizar el proceso de desarrollo. Mantener registros cuidadosos es esencial para sistemas grandes y complejos desarrollados desde diferentes ubicaciones, pero innecesario en proyectos pequeños.

Planificación de la gestión de configuraciones

Un plan de gestión de configuraciones describe los estándares y procedimientos utilizados para la gestión de la configuración. El punto de inicio para desarrollar el plan es un conjunto de estándares generales de gestión de la configuración de toda la compañía adaptables a cada proyecto específico.

Una parte importante del plan de la CM es la definición de responsabilidades. Define quién es el responsable de la entrega de cada documento o de cada componente de software para la garantía de la calidad y la gestión de las configuraciones. También define los revisores de cada documento. La persona responsable de la entrega de los documentos no es preciso que sea la misma responsable de producir el documento. Para simplificar las interfaces, a menudo es conveniente hacer que los gestores de proyectos o los líderes del equipo sean responsables de todos los documentos producidos por su equipo.

Durante el proceso de planificación de la gestión de configuraciones, se decide exactamente qué elementos (o clases de elementos) se van a controlar. Los documentos o grupos de documentos relacionados del control de la configuración son documentos formales o elementos de la configuración. Normalmente, los planes del proyecto, las especificaciones, los diseños, los programas y los conjuntos de datos de prueba son elementos de la configuración. Todos los documentos que son necesarios para el mantenimiento futuro del sistema deben ser controlados por el sistema de control de configuraciones.

La base de datos de configuraciones

La base de datos de configuraciones se utiliza para registrar toda la información relacionada con las configuraciones y sus elementos. Sus funciones principales son ayudar a la evaluación del impacto de los cambios en el sistema y proveer información de la gestión acerca del proceso de la CM. Además de definir el esquema de la base de datos de la configuración, como parte del proceso de planificación de la CM también se deben definir los formularios y los procedimientos para registrar y recuperar la información del proyecto.

De forma ideal, la base de datos de configuraciones se integra en el sistema de gestión de las versiones utilizado para almacenar y gestionar los documentos formales del proyecto. Este enfoque, apoyado por algunas herramientas CASE integradas, hace posible vincular los cambios de forma directa con los documentos y componentes afectados por el cambio. Se da mantenimiento a los vínculos entre los documentos, como los documentos de diseño y el código del programa, con el fin de que sea relativamente fácil encontrar todo lo que debe modificarse cuando se propone un cambio.

Gestión del cambio

Los procedimientos de gestión de cambios se ocupan del análisis de costes y beneficios de los cambios propuestos, aprobando aquellos cambios que merecen la pena y registrando los componentes del sistema que se tienen que cambiar. El proceso de gestión de cambios (véase la Figura 29.3) se lleva a cabo cuando el software o la documentación asociada se pone bajo el control del equipo de gestión de configuraciones.

La primera etapa del proceso de gestión del cambio es completar un formulario de solicitud de cambios (CRF) en el cual el solicitante señala los cambios requeridos en el sistema. Además de registrar los cambios requeridos, el CRF registra las recomendaciones pertinentes al cambio, los costes estimados del cambio y las fechas en las que se solicita, prueba, implementa y valida el cambio. También incluye una sección donde el analista señala cómo implementar el cambio.

Una vez emitido el formulario, se registra en la base de datos de configuraciones. Se analiza la petición de cambios para comprobar que el cambio solicitado es necesario. Algunas peticiones de cambio se deben a los malentendidos más que a los fallos del sistema y, por lo tanto, no es necesario modificar el sistema. Otras se refieren a fallos ya conocidos. Si el análisis descubre que el cambio solicitado es inválido, está duplicado o ya ha sido considerado, el cambio se rechaza. La razón del rechazo se devuelve a la persona que emitió la solicitud del cambio.

El consejo de control de cambios (CCB) revisa y aprueba todas las peticiones de cambios a menos que el cambio simplemente comprenda la corrección de errores menores en los despliegues de la pantalla, las páginas web o en los documentos.

La denominación «consejo de control de cambios» implica que un grupo toma las decisiones del cambio. Los proyectos militares requieren consejos de control de cambios estructurados formalmente, que incluyen a los clientes y los proveedores. Sin embargo, para proyectos pequeños o de medio tamaño, el consejo de control de cambios simplemente se compone de un gestor de proyectos más uno o dos ingenieros.

Las peticiones de cambio están generalmente asociadas con errores en el sistema que han sido descubiertos durante las pruebas del sistema o por nuestros clientes una vez que ha sido entregado el mismo. Los clientes pueden enviar los informes de error a través de una página web o por e-mail. Un equipo de gestión de errores comprobará que los errores remitidos son válidos y los traducirá a peticiones de cambio formales del sistema. Los cambios tienen que ser priorizados para su implementación y los errores no pueden ser reparados si los costes de reparación son muy elevados.

En algunos métodos ágiles, como programación extrema, los clientes son directamente implicados en decidir cuándo un cambio debe ser implementado. Cuando ellos proponen un cambio en los requerimientos del sistema, trabajan con el equipo en evaluar el impacto del cambio y deciden cuándo deben tener prioridad respecto a los planes establecidos para el próximo incremento del sistema.

Gestión de versiones y entregas

La gestión de las versiones y entregas es el proceso de identificar y mantener los registros de las diversas versiones y entregas de un sistema. Los gestores de las versiones diseñan procedimientos para asegurar que las diversas versiones de un sistema se puedan recuperar cuando se requieran y que no se cambien de forma accidental

Una versión de un sistema es una instancia de un sistema que difiere, de alguna manera, de otras instancias. Las nuevas versiones de un sistema tienen diferente funcionalidad, mejor rendimiento o incorporan reparaciones de los fallos del sistema

Una entrega de un sistema es una versión que se distribuye a los clientes. Cada entrega incluye nueva funcionalidad o está concebida para diferentes plataformas de hardware. Siempre existen más versiones de un sistema que las entregas, puesto que las versiones se crean dentro de una organización para el desarrollo interno o pruebas y nunca se entregan a los clientes.

Gestión de entregas

Una entrega del sistema es una versión del sistema que se distribuye a los clientes. Los gestores de entregas del sistema son los responsables de decidir cuándo se entrega el sistema, de gestionar el proceso de creación de las entregas y de los medios de distribución y documentación de la entrega para asegurar que se puedan recuperar de la misma forma en que se distribuyeron, en Caso necesario.

Archivos de configuración

Los archivos de datos necesarios

El programa de instalación

La documentación electrónica

El embalaje y la publicidad asociados diseñados para esta entrega.

Herramientas CASE para gestión de configuraciones

Los procesos de gestión de configuraciones por lo general están estandarizados e involucran la aplicación de procedimientos predefinidos. Requieren la gestión cuidadosa de grandes cantidades de datos, y la atención a los detalles es esencial. Cuando se construye un sistema a partir de las versiones de los componentes, un simple error en la gestión de configuraciones puede implicar que el software no trabaje de forma adecuada. En consecuencia, las herramientas CASE de apoyo son esenciales para la gestión de configuraciones, y desde los 70 se ha producido un gran número de diferentes herramientas que abordan diferentes áreas de la gestión de configuraciones.

Dos tipos de entornos de trabajo CM

Entornos de trabajo abiertos. Las herramientas para cada etapa del proceso CM son integradas de acuerdo con procedimientos organizacionales estándar. Hay muchas herramientas CM comerciales

y open-source disponibles para propósitos específicos. La gestión de cambios puede llevarse a cabo con herramientas de seguimiento, la gestión de versiones

Entornos integrados. Estos entornos ofrecen facilidades integradas para gestión de versiones, construcción del sistema o seguimiento de los cambios. Las ventajas de los entornos CM integrados es que el intercambio de datos es sencillo, y que el entorno incluye una base de datos CM integrada.

Soporte para gestión de versiones

Cada persona involucrada en el proceso de gestión de cambios es responsable de alguna actividad. Una vez que completa esta actividad pasa los formularios y elementos de configuración asociados a otra persona. La naturaleza de procedimiento de este proceso significa que un cambio en el modelo del proceso se diseña e integra con un sistema de gestión de versiones. Entonces este modelo se interpreta para que los documentos correctos se pasen a la persona indicada en el momento justo.

Hay varias herramientas de gestión de cambios disponibles. desde herramientas relativamente simples. Estas herramientas proveen alguna o todas de las siguientes características de apoyo al proceso:

Un editor de formularios que permite cambiar los formularios propuestos a crear y rellenar por las personas que hacen las peticiones de cambios.

Un sistema de flujo de trabajo que permiten al equipo de la CM especificar las personas que deben procesar las solicitudes de peticiones de cambio y el orden del procesamiento.

Una base de datos de cambios que se utiliza para gestionar todas las propuestas de cambios y que puede vincularse al sistema de gestión de versiones. Por lo general, se proveen las características de consulta que permiten al equipo de la CM encontrar propuestas específicas de cambios.

Un sistema de gestión de informes de cambios que genera informes sobre el estado de las peticiones de cambio recibidas.

Auditoría de configuración

La identificación, control de versión y control del cambio ayudan a conservar el orden en lo que de otro modo sería una situación caótica y fluida. Sin embargo, incluso los más exitosos mecanismos de control rastrean un cambio sólo hasta que se genera una **OCI (Orden de Cambio de Ingeniería)**. ¿Cómo puede un equipo de software asegurarse de que el cambio se implementó adecuadamente? La respuesta es doble: 1) revisiones técnicas y 2) auditoría a la configuración del software.

La revisión técnica (capítulo 15) se enfoca en la exactitud técnica del objeto de configuración que se modificó. Los revisores valoran el ICS para determinar la consistencia con otros ICS, así como omisiones o potenciales efectos colaterales. Una revisión técnica debe realizarse para todos los cambios, salvo los más triviales.

Una auditoría de configuración del software complementa la revisión técnica al valorar un objeto de configuración acerca de las características que por lo general no se consideran durante la revisión.

1. ¿Se realizó el cambio especificado en la OCI? ¿Se incorporó alguna modificación adicional?
2. ¿Se llevó a cabo una revisión técnica para valorar la exactitud técnica?
3. ¿Se siguió el proceso del software y se aplicaron adecuadamente los estándares de ingeniería de software?
4. El cambio se "resaltó" en el ICS? ¿Se especificaron la fecha del cambio y el autor del cambio? ¿Los atributos del objeto de configuración reflejan el cambio?
5. ¿Se siguieron los procedimientos ACS para anotar, registrar y reportar el cambio?
6. ¿Los ICS relacionados se actualizaron adecuadamente?

Identificación de los Riesgos

Los riesgos del proyecto amenazan el plan del proyecto, es decir, si los riesgos del proyecto se vuelven reales, es probable que el calendario del proyecto se deslice y que los costos aumenten. Los riesgos del proyecto

identifican potenciales problemas de presupuesto, calendario, personal (tanto técnico como en la organización), recursos, participantes y requisitos, así como su impacto sobre un proyecto de software

Los riesgos técnicos amenazan la calidad y temporalidad del software que se va a producir. Si un riesgo técnico se vuelve una realidad, la implementación puede volverse difícil o imposible.

Los riesgos empresariales amenazan la viabilidad del software que se va a construir y con frecuencia ponen en peligro el proyecto o el producto.

La identificación de riesgos es un intento sistemático por especificar amenazas al plan del proyecto (estimaciones, calendario, carga de recursos, etc.). Al identificar los riesgos conocidos y predecibles, el gerente de proyecto da un primer paso para evitarlos cuando es posible y para controlarlos cuando es necesario.

Existen dos tipos distintos de riesgos para cada una de las categorías que se presentaron en la sección: riesgos genéricos y riesgos específicos del producto. Los riesgos genéricos son una amenaza potencial a todo proyecto de software. Los riesgos específicos del producto pueden identificarse solamente por quienes tienen clara comprensión de la tecnología, el personal y el entorno específico del software que se construye. Para identificar los riesgos específicos del producto, examine el plan del proyecto y el enunciado de ámbito del software, y desarrolle una respuesta a la siguiente pregunta: ¿qué características especiales de este producto pueden amenazar el plan del proyecto?

PROYECCIÓN DEL RIESGO

La proyección del riesgo, también llamada estimación del riesgo intenta calificar cada riesgo en dos formas: 1) la posibilidad o probabilidad de que el riesgo sea real y 2) las consecuencias de los problemas asociados con el riesgo, en caso de que ocurra. Usted trabaja junto con otros gerentes y personal técnico para realizar cuatro pasos de proyección de riesgo:

1. Establecer una escala que refleje la probabilidad percibida de un riesgo.
2. Delinear las consecuencias del riesgo.
3. Estimar el impacto del riesgo sobre el proyecto y el producto.
4. Valorar la precisión global de la proyección del riesgo de modo que no habrá malentendidos.

REFINAMIENTO DEL RIESGO

Durante las primeras etapas de la planificación del proyecto, un riesgo puede enunciarse de manera muy general. Conforme pasa el tiempo y se aprende más acerca del proyecto y de los riesgos, es posible refinar el riesgo en un conjunto de riesgos más detallados, cada uno un poco más sencillo de mitigar, monitorear y manejar

MITIGACIÓN, MONITOREO Y MANEJO DE RIESGO

Si un equipo de software adopta un enfoque proactivo ante el riesgo, evitarlo siempre es la mejor estrategia. Esto se logra desarrollando un plan para mitigación del riesgo

Conforme avanza el proyecto, comienzan las actividades de monitoreo de riesgos. El gerente de proyecto monitorea factores que pueden proporcionar un indicio de si el riesgo se vuelve más o menos probable. Además de monitorear dichos factores, un gerente de proyecto debe dar seguimiento a la efectividad de los pasos de mitigación del riesgo. Por ejemplo, un paso de mitigación del riesgo anotado aquí requiere la definición de estándares de producto operativo y mecanismos para asegurarse de que los productos operativos se desarrollan en forma oportuna.

El manejo del riesgo y la planificación de contingencia suponen que los esfuerzos de mitigación fracasaron y que el riesgo se convirtió en realidad. Continuando con el ejemplo, el proyecto ya está en marcha y algunas personas anuncian que renunciarán al mismo. Si se siguió la estrategia de mitigación, está disponible el respaldo, la información se documentó y el conocimiento se dispersó a través del equipo.

En el plan de proyecto del software puede incluirse una estrategia de administración del riesgo, o los pasos de administración del riesgo pueden organizarse en un plan de mitigación, monitoreo y manejo de riesgo (MMMR) por separado. El plan MMMR documenta todo el trabajo realizado como parte del análisis de riesgos y el gerente del proyecto lo usa como parte del plan de proyecto global.

Estándares

IEEE 828-2012: Es el estándar más específico para la Gestión de Configuración en Ingeniería de Sistemas y Software. Define las actividades mínimas exigibles, desde la identificación hasta la auditoría.

ISO/IEC/IEEE 12207: Estándar del ciclo de vida del software que incluye a la Gestión de Configuración como uno de los procesos de soporte fundamentales.

ITIL (SACM): En el marco de la gestión de servicios, el *Service Asset and Configuration Management* se enfoca en controlar los activos de servicio y los ítems de configuración (CIs) necesarios para entregar un servicio de TI.

CMMI (Capability Maturity Model Integration): Define la gestión de configuración como un área de proceso clave para alcanzar niveles de madurez organizativa.

Gestión de Dependencias: Usar herramientas como Maven (Java), npm (Node.js) o NuGet (C#) para asegurar que todos los desarrolladores usen exactamente las mismas versiones de las librerías externas.

Integración Continua (CI/CD): Automatizar el proceso de construcción y prueba cada vez que se detecta un cambio en el repositorio, asegurando que la configuración sea estable en todo momento.

Integración Continua (CI - Continuous Integration)

La **CI** se centra en la fase inicial del desarrollo. Consiste en que los desarrolladores suban sus cambios al repositorio central (como Git) de forma frecuente (varias veces al día).

- **Proceso:** Cada vez que se sube código, un servidor de CI (como Jenkins o GitHub Actions) compila el proyecto y ejecuta pruebas automatizadas.
- **Objetivo:** Detectar errores de integración lo antes posible. Si el código de un programador "rompe" el de otro, el sistema avisa de inmediato.
- **Requisito clave:** Tener una buena cobertura de **pruebas unitarias** y de integración.

Entrega y Despliegue Continuo (CD)

Entrega Continua (Continuous Delivery)

El software está siempre en un estado en el que **podría** ser desplegado en producción en cualquier momento.

- **Flujo:** Después de pasar las pruebas de CI, el código se despliega automáticamente en un entorno de pruebas (Staging/QA).
- **Intervención humana:** El paso final a la infraestructura de producción requiere una **aprobación manual** (un "botón" que alguien presiona).
- **Despliegue Continuo (Continuous Deployment)**

Es el nivel más avanzado. No hay intervención humana entre la escritura del código y su llegada al usuario final.

- **Flujo:** Si el código pasa todas las pruebas automatizadas en la tubería (pipeline), se despliega **directamente en producción**.
- **Riesgo:** Requiere una confianza absoluta en la suite de pruebas y sistemas de monitoreo automáticos para revertir cambios (rollback) si algo sale mal.

